Technische Hochschule Deggendorf
Fakultät Angewandte Informatik
Studiengang Artificial Intelligence and Data Science

Zero-Shot-Assoziation von Nährstoffen und Mengen mittels OCR-gestütztem semantischem Graph-Matching

Zero-Shot Nutrient and Quantity Association using OCR-Guided Semantic Graph Matching

Masterarbeit zur Erlangung des akademischen Grades:
Master of Science (M.Sc.)
an der Technischen Hochschule Deggendorf

Vorgelegt von:
Moustafa Hamada
Matrikelnummer: 12401478

Prüfungsleitung:
Prof. Dr. [Supervisor Name]
Technische Hochschule Deggendorf

Am: September 2026

Erklärung

Name des Studierenden: Moustafa Hamada

Name des Betreuenden: Prof. Dr. [Supervisor Name]
Technische Hochschule Deggendorf

Thema der Abschlussarbeit:

Zero-Shot-Assoziation von Nährstoffen und Mengen mittels OCR-gestütztem semantischem Graph-Matching . . . . . .

. . . . . .

. . . . . .

. . . . . .

1. Ich erkläre hiermit, dass ich die Abschlussarbeit gemäß § 35 Abs. 7 RaPO (Rahmenprüfungsordnung für die Fachhochschulen in Bayern, BayRS 2210-4-1-4-1-WFK) selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

   Deggendorf, ...
   Datum Unterschrift des Studierenden

2. Ich bin damit einverstanden, dass die von mir angefertigte Abschlussarbeit über die Bibliothek der Hochschule einer breiteren Öffentlichkeit zugänglich gemacht wird:

   ◯ Nein

   ◯ Ja, nach Abschluss des Prüfungsverfahrens

   ◯ Ja, nach Ablauf einer Sperrfrist von ...Jahren.

   Deggendorf, ...
   Datum Unterschrift des Studierenden

Bei Einverständnis des Verfassenden vom Betreuenden auszufüllen:

Eine Aufnahme eines Exemplars der Abschlussarbeit in den Bestand der Bibliothek und die Ausleihe des Exemplars wird:

◯ Befürwortet

◯ Nicht befürwortet

Deggendorf, ...
Datum

. . . . . .
Unterschrift des Betreuenden

Contents

1 Background **1**

1.1 Problem Framing and Pipeline Commitments . . . 1
1.2 Optical Character Recognition and Post-OCR Repair . . . 3
1.2.1 OCR as the Perceptual Entry Point . . . 3
1.2.2 The Two-Stage Paradigm . . . 4
1.2.3 OCR Error Taxonomy . . . 5
1.2.4 Post-OCR Correction and Text Normalisation . . . 6
1.3 Document Layout and Geometric Structure . . . 7
1.3.1 Bounding Boxes and Reading Order . . . 7
1.3.2 Columns, Rows, and Headers . . . 7
1.3.3 Geometric Distortions . . . 8
1.3.4 Context-Header Attachment Under Column Slope . . . 9
1.4 Token Classification and Named Entity Recognition . . . 10
1.4.1 The Classifier-Stage Design Space . . . 10
1.4.2 Rule-Based Classification . . . 11
1.4.3 Learned NER as a Classifier Alternative . . . 12
1.4.4 Embedding-Based Classification (Brief Overview) . . . 13
1.4.5 Three Faces of Zero-Shot Classification . . . 13
1.5 Graph Representations of Documents . . . 14
1.5.1 Graphs as a Document Representation . . . 14
1.5.2 Edge Types and Soft Adjacency . . . 15
1.5.3 Tail-Collision Avoidance . . . 16
1.5.4 Bipartite Matching, with Rank-Under-Context as Fallback . . . 17
1.5.5 Deterministic versus Learned Graph Reasoning . . . 19
1.6 Embeddings and Semantic Similarity . . . 20
1.6.1 Embeddings: From Word Vectors to Multilingual Sentence Encoders . 20
1.6.2 BGE-M3 and Cosine Similarity . . . 21
1.6.3 Prototype Matching, Thresholds, and Margins . . . 21
1.6.4 Failure Modes of Prototype Matching . . . 23
1.7 Evaluation . . . 24
1.7.1 Precision, Recall, and F1 . . . 24
1.7.2 Token-, Span-, and Tuple-Level Predicates . . . 24
1.7.3 The 4-Field Tuple F1 of This Thesis . . . 25
1.7.4 Normalisation and the Two-Tier Evaluator . . . 26
1.7.5 Per-Image Variance and Diagnostic Logging . . . 27
1.8 Modular Pipeline Design Philosophy . . . 27

2 Related Work **29**
2.1 Supervised document information extraction and benchmark-driven progress 30
2.1.1 The document-IE problem and its benchmark substrate 30
2.1.2 The supervised paradigm and what it assumes 31
2.1.3 Where this thesis sits relative to the supervised regime 32
2.2 Table structure recognition 33
2.2.1 The table structure recognition problem and its representations 33
2.2.2 Representative TSR systems and their architectural commitments . . . 33
2.2.3 Why this thesis does not include a TSR stage 34
2.3 OCR-free and vision-language document understanding 35
2.3.1 From OCR-dependent pipelines to OCR-free document models 35
2.3.2 Representative OCR-free and vision-language systems 36
2.3.3 Why this thesis retains explicit OCR and modular extraction 37
2.4 Graph-based key information extraction 39
2.4.1 Graph-based KIE as a research line 39
2.4.2 Representative graph-based KIE systems: PICK, SPADE, and FormNet 39
2.4.3 Where this thesis sits: deterministic graph association without supervised graph learning 40
2.5 Zero-shot information extraction and semantic matching 41
2.5.1 Zero-shot IE as a research line 41
2.5.2 Representative zero-shot IE families 42
2.5.3 Where this thesis sits 43
2.6 Nutrition-label and food-data extraction 44
2.6.1 Prior work specifically on nutrition-label extraction 44
2.6.2 Adjacent food-data and retail-packaging extraction 45
2.6.3 The gap this thesis addresses 45
2.7 Positioning of this thesis 46

1 Background

This chapter establishes the technical foundation that the rest of the thesis builds on. The pipeline of this thesis transforms images of supplement nutrition labels into structured records, and that transformation depends on a specific stack of concepts and primitives — optical character recognition, document geometry, token classification, graph reasoning, semantic embeddings, and a quantitative evaluation regime — each of which has its own history in the document-understanding and natural-language-processing literatures. The chapter introduces these concepts at the level of detail required to follow the methodology and experimental chapters that come next, in the order in which the pipeline encounters them. Positioning of the thesis against specific published systems and against the broader research lineage is the work of Chapter 2; Chapter 1 confines itself to the conceptual machinery the pipeline depends on.

The chapter is organised as follows. §1.1 frames the extraction-and-association problem the pipeline solves and states the three commitments — zero-shot, layout-flexible within the supplement-label domain, and modular and interpretable — that the rest of the thesis is structured around. §1.2 covers optical character recognition and the post-OCR repair stage that follows. §1.3 introduces the geometric primitives — bounding boxes, columns, rows, headers, and the formalism the pipeline uses to handle column slope. §1.4 surveys the three classifier families the thesis evaluated as candidate implementations of the classification stage. §1.5 develops the graph representation that the association stage operates on, including the typed edges, the tail-collision-avoidance step, the bipartite matching procedure, and the rank-under-context fallback. §1.6 deepens the treatment of embedding-based classification specifically, introducing the multilingual sentence encoder used in the embedding ablation. §1.7 defines the evaluation regime, the four-field tuple metric, and the two-tier matching protocol the experimental chapter relies on. §1.8 closes by articulating the architectural philosophy that runs through the chapter as a whole.

1.1 Problem Framing and Pipeline Commitments

The pipeline of this thesis takes an image of a supplement nutrition label as input and produces structured tuples as output. The label may be presented in any of the layouts encountered in the European supplement market: a flat table, a multi-column panel, a skewed or curved label, or a paragraph-style declaration in which the values are embedded in flowing prose. The pipeline must solve two problems jointly. The first is *extraction*: recovering nutrient names, numeric values, units of measurement, and reference-base contexts from the image text. The second is *association*: determining which nutrient each value belongs to, which unit accompanies which value, and which reference base each pairing is reported under. The output is a structured record of the form $\langle \text{nutrient}, \text{quantity}, \text{unit}, \text{context} \rangle$, where the *nutrient* names what is being measured, the *quantity* and *unit* give the numerical value and its unit of measurement, and

the *context* names the reference base on which the value is stated (per 100 g, per serving, per daily dose). A tuple counts as correct only when extraction has succeeded for all four fields and association has linked them to the same underlying fact on the label. The pipeline is built for this domain — it is not a general document extraction system — but within the domain it is designed to handle the full range of layouts that commercial supplement labelling presents.

Supplement labels are visually small, information-dense, and idiosyncratic, and the combination of these properties makes them a harder extraction target than the receipts and forms that dominate the document-AI literature. Five properties recur. **Dense numeric content**: a typical label encodes between eight and forty quantitative facts in a single rectangular region a few square centimetres in size, and adjacent rows differ only by a single nutrient name and a small number, so a one-row error in association destroys the tuple. **Multi-context columns**: EU labels frequently report nutrients twice in adjacent columns, once per 100 g and once per serving, and the two columns of identical numbers must be distinguished only by their column header, which sits several rows away from the data it governs. **Multilinguality**: EU directive 1169/2011 permits labels in multiple official languages on the same package, and slash-separated polyglot rows such as *Eiweiß / Protein / Protéines* appear routinely on products sold across the European market. **OCR-hostile typography**: labels are printed in small fonts (often below 6 pt for non-mandatory information) on curved or wrinkled plastic surfaces, and feature dense table borders that confuse line-detection heuristics. **Sub-nutrient hierarchy**: rows such as *davon gesättigte Fettsäuren* (of which: saturated fats) are conceptually a sub-component of the row above them, but they appear at the same visual indentation as their parent, and a pipeline that reasons only on visual layout will treat them as independent peer nutrients rather than as nested under their parent. A sixth and qualitatively different case is **non-tabular labels**, in which the values are not arranged in a grid at all but are embedded in flowing prose: "Per 100 g: 1500 kJ energy, 12 g protein, 45 g carbohydrates of which 8 g sugars, ...". A pipeline restricted to tabular reasoning fails entirely on these labels.

The first commitment is **zero-shot**. The pipeline does not use labelled examples of the target task to train, fit, or tune any component. Lexicons, rules, and decision logic are written from domain knowledge — the EU 1169/2011 nutrient-name vocabulary, the standard units of measurement, the canonical reference-base context labels — before any image is processed. Labelled data is used only for evaluation, to measure how well the pipeline performs against ground truth, and is never consulted during inference. The single exception is threshold selection for the embedding-based classifier, where a held-out subset of labelled data is used to choose decision thresholds via grid search; this follows standard practice for prototype-based zero-shot classifiers and does not constitute training in the conventional sense, since no model parameters are fit. The cost of this commitment is that the pipeline cannot learn from its own mistakes — every failure mode must be diagnosed by hand and addressed by changing rules, lexicons, or geometric primitives, with no gradient signal to absorb the error. The benefit is that the pipeline generalises to any supplement label without re-annotation: a new product, a new layout, a new language combination is handled by the same code that handled the labelled evaluation set, with no retraining cycle. This generalisation is what makes the pipeline deployable at the scale of the European supplement market, where tens of thousands of products with new layouts enter circulation each year.

The second commitment is that the pipeline must **work across the full range of layouts**

that supplement labels exhibit, without layout-specific code paths. Three families of mechanisms deliver this commitment together. *Geometric mechanisms* — adaptive row thresholding, column-derived direction inference, and column-role inference — handle the typical case in which the layout is recoverable from local geometry. *Structural mechanisms* — rank-under-context row membership and translation-row collapsing — recover cases where local geometry is unreliable, by reasoning about which token holds which structural position within its column rather than by measuring pixel distances. *Surface-form fallback* — a paragraph-level extractor — handles the case where no tabular structure is recoverable at all, taking over when row induction fails or when the structured path produces no output. The cost of this commitment is that no single mechanism can be tuned to optimum: each must work well enough on the inputs it sees so that the family below it is not invoked unnecessarily, and the architecture is more involved than a layout-specific pipeline would be. The benefit is that the pipeline does not fail catastrophically on unfamiliar layouts: a curved label, a multi-column panel, a paragraph-style declaration, or a label with unusual row spacing is handled by the same code that handles the typical case, because the typical case is just one point in a continuum the pipeline already covers.

The third commitment is that the pipeline must be **modular and interpretable**. The pipeline is organised as a sequence of independent stages: optical character recognition, post-OCR correction, semantic classification and token enrichment, graph construction, tuple association, and evaluation. Each stage has a typed input–output contract: it receives a fixed schema from the stage above it and emits a fixed schema to the stage below, and no stage depends on the internal implementation of another. This permits, for example, swapping the OCR engine without touching the classifier, or replacing the rule-based classifier with an embedding-based one while keeping the graph unchanged. Each stage also emits a diagnostic record that captures its internal decisions: how many tokens it received, how many it dropped, which correction rules fired, which classifier fallback was used, what edges the graph constructor pruned, why a given association was made. The diagnostic records concatenate into a per-image audit trail in which every output tuple can be traced backwards to the OCR tokens that produced it. The cost of this commitment is engineering overhead: each stage must define and uphold its schema, the diagnostic logging adds to the codebase, and a single global optimisation across stages is not available because the stages cannot reach into one another. The benefit is that failure modes are localisable: a regression on a single image can be attributed to a specific stage, a specific token, and a specific decision, which is what makes single-variable ablations a viable methodology and what allows the experimental chapter to make stage-level causal claims of the form "swapping the OCR engine raised nutrient F1 by N percentage points" with no other change in the pipeline.

1.2 Optical Character Recognition and Post-OCR Repair

1.2.1 OCR as the Perceptual Entry Point

Optical Character Recognition is the perceptual entry point of the pipeline. It is the only stage that operates on pixel data: every later stage consumes its output and never sees the original image. This places OCR in a distinctive position relative to the rest of the pipeline. Recognition

errors at this stage are unrecoverable in a strict sense — no downstream component, however carefully engineered, can reconstruct a token that OCR failed to detect. A misread numeric value propagates through the graph unchanged, and a missing nutrient name simply does not appear in the output. The perceptual layer therefore sets the ceiling on what every subsequent stage can achieve, which is why the choice of OCR engine and the post-OCR repair stage that immediately follows are treated as primary determinants of pipeline performance rather than as routine preprocessing.

1.2.2 The Two-Stage Paradigm

Modern OCR pipelines often decompose text extraction into two cascaded sub-problems: text detection and text recognition [1]. The detection stage takes a document image and produces a set of polygons — usually quadrilaterals or axis-aligned rectangles — that localise text-bearing regions. The recognition stage then takes each cropped region and transcribes it into a string. The factorisation is attractive for two reasons. It permits independent optimisation of the spatial and the lexical components, and the per-region crop removes most of the layout variability before recognition begins, since the recogniser sees often one word or line crop at a time rather than a whole page.

Three influential families of detection architectures are **CRAFT** [2], which predicts a per-pixel region score together with an affinity score between adjacent characters and produces character-level boxes that can be merged into word-level polygons; **DBNet** [3], which integrates the binarisation step directly into the network so that the threshold itself is a learned function of position, giving sharper segmentation maps on dense text; and **EAST** [4], which regresses quadrilaterals from a single fully convolutional network in one shot and supports rotated boxes, trading some boundary precision for substantially faster inference. Other detectors used in production OCR systems take different approaches — progressive scale expansion (PSENet), pixel aggregation (PAN), transformer-based regression — but the three above are the most influential references in the line that leads to the OCR engines this thesis evaluates. The choice between detector families primarily trades off polygon quality, robustness to curvature, and speed; supplement labels, with their dense rows and small fonts, benefit most from the higher-precision boundaries of CRAFT and DBNet.

Recognition transduces a cropped region image into a character string. The dominant architecture for almost a decade has been the Convolutional Recurrent Neural Network (**CRNN**) [5]: a CNN encoder feeds a bidirectional LSTM, which feeds a Connectionist Temporal Classification (CTC) head [6] that aligns variable-length predictions with the input without requiring per-character supervision. The CTC loss marginalises over possible alignments by introducing a special blank symbol and a many-to-one mapping that collapses repeated symbols and removes blanks; this marginalisation makes CTC robust to local uncertainty in character alignment, but it does not explicitly model two-dimensional layout or long-range semantic structure, and so CRNN-class recognisers can become fragile when the upstream detector provides crops that contain multi-line, multilingual, or table-like content. More recent recognisers replace the BiLSTM with attention decoders or with transformer-based modules — examples include the Neural-machine-translation-style Reading Transformer (NRTR) [7], which uses a self-attention encoder–decoder without recurrence, and the Single Visual model for scene Text Recognition

(SVTR) [8], which abandons the recurrent module entirely in favour of patch-wise visual mixing. Hybrid recognisers run an attention branch and a CTC branch in parallel and either ensemble or distil between them. These attention-based recognisers can handle longer-range dependencies more flexibly than pure CTC, although multi-line document crops still require appropriate detection, rectification, or layout-aware modelling at the surrounding stages of the pipeline.

1.2.3 OCR Error Taxonomy

The output of an OCR engine on supplement labels is not error-free text but text with a characteristic distribution of errors, and the post-OCR repair stage that follows is designed against this distribution. Five categories recur across engines and across labels.

Character confusion is the substitution of one character for another with similar shape. The canonical examples are `0` for `O`, `l` for `1`, `rn` for `m`, and μ for `u`, the last of which is particularly damaging on the unit field because μg (micrograms) and mg (milligrams) differ by three orders of magnitude. Character confusion is most damaging on numeric tokens, where a single substitution flips the value, and on unit tokens, where the unit dictates the magnitude of the entire fact.

Token fusion is the failure to insert a whitespace boundary between adjacent units of meaning. A label printed as `400 mg` may be returned as the single token `400mg` when the detection stage produces a polygon that spans both the value and the unit. The downstream consequence is severe: the classifier may see one ambiguous token, the unit is never extracted as a separate node, and the tuple loses its unit field entirely.

Row merging is the dual failure: the detection stage groups two visually adjacent rows into a single text line, and the recogniser transcribes them as one string. The result is a token like `Eiweiß 12 g Fett 8 g` in which two complete nutrient–quantity–unit triples appear in a single OCR cell. A purely lexical rule can split such a token into substrings, but it cannot reliably recover which substring belongs to which row of the original label without spatial information from the bounding boxes. The row-merging case is one of the strongest arguments for a graph-based association stage that operates on geometry rather than on the raw token list.

Non-nutritional text noise arises when the detection stage produces line crops that span an ingredient list, a marketing claim, a regulatory disclaimer, or other prose unrelated to the nutritional declaration. The crop may also contain a mixture of languages, since EU labels are routinely multilingual. The recogniser receives input that is poorly aligned with the target extraction task and produces a low-confidence transcription with garbled segments and dropped tokens. The downstream effect is that some images return a token list dominated by NOISE-classified tokens with no recoverable nutritional content; these are the OCR-failure cases the experimental chapter discusses.

Artefact injection is the appearance of stray punctuation or formatting characters at the boundaries of otherwise correct tokens. Common artefacts include leading single quotes or square brackets, trailing pipes, and vertical bars used as table-cell separators that the recogniser misreads as `|` or `}` characters; these prevent lexicon matching even when the underlying nutrient name is recognised correctly. The corrector strips them as a deterministic preprocess-

ing step before the classifier sees the token stream.

1.2.4 Post-OCR Correction and Text Normalisation

Two deterministic text-transformation layers sit between the OCR output and the rest of the pipeline. The first is **post-OCR correction**, which attempts to repair the errors of §1.2.3. The second is **text normalisation**, which maps valid surface forms to canonical representations. The two layers are conceptually distinct: post-OCR correction targets likely OCR-induced errors, while normalisation maps valid surface variation to canonical forms. Both operate by deterministic rules rather than learned parameters, but their inputs and outputs and their reasons for existing are different.

Post-OCR correction in this thesis is implemented as an ordered sequence of rules applied to the raw OCR token list. The rules target the failure categories from §1.2.3 directly: token-fusion rules split a single OCR token into multiple tokens when the token contains both a numeric value and an alphabetic unit (`400mg` is split into `400` and `mg`); regulatory-string rules strip non-nutritional substrings such as Nutrient Reference Value percentages that recur in standard EU label templates and would otherwise enter the classifier as spurious nutrient or context candidates; artefact-stripping rules remove leading and trailing punctuation that prevents lexicon matching. A small number of rules also merge tokens when an OCR-introduced whitespace has separated a name that should be interpreted as one semantic unit; in such cases the merged token is given a combined bounding box that is the union of its constituents, so that downstream stages reasoning about geometry continue to receive a single coherent spatial object. Each rule is a small deterministic transformation derived from inspection of representative failure cases on a development subset, and the rule set is extended by adding new rules rather than by retraining a model. The corrector emits a per-image log of which rules fired on which tokens, which feeds the diagnostic audit trail of §1.7.5.

Text normalisation operates at two distinct points in the pipeline, with different roles. The first is at **inference time**, inside the classifier, where surface variants of the canonical context labels are mapped onto a closed set of canonical values: strings such as `PRO PORTION`, `Je 100g`, `pro Portion`, and `per 100 g` are collapsed onto canonical labels including `per_100g`, `per_100ml`, `per_serving`, and `per_daily_dose`. This mapping is necessary for the pipeline to produce stable context-field values across labels written in different languages and styles, and it is consulted by the classifier on every token classified as a context candidate. The second is at **evaluation time**, inside the matching predicates of §1.7.3, where additional normalisations are applied to the predicted and ground-truth strings before comparison: lowercase folding, German umlaut transliteration, diacritic stripping, decimal-comma standardisation. The two normalisation steps are deliberately separate. Inference-time normalisation produces the canonical output the pipeline commits to. Evaluation-time normalisation is a comparison aid that allows two strings already correct in their respective canonical forms to be recognised as equal despite incidental representational differences.

Keeping post-OCR correction and text normalisation as separate layers serves two purposes. The boundary between them is the boundary between repairing likely OCR errors and standardising valid surface variation, and a pipeline that conflates the two tends to over-correct: a single normalisation rule applied indiscriminately to OCR output may silently rewrite genuinely

incorrect tokens into plausible-looking but wrong canonical forms. Keeping the layers distinct also keeps each one diagnosable. The corrector log records which OCR errors were repaired and which slipped through; the inference-time normaliser records which surface variants were canonicalised and which were left unrecognised; the evaluation-time normaliser is part of the metric definition, not part of the extraction pipeline itself. A failure in any one of these can be traced to its layer without disturbing the others.

1.3 Document Layout and Geometric Structure

1.3.1 Bounding Boxes and Reading Order

After OCR and post-OCR repair, the pipeline represents each token as text plus geometry. The geometry locates the token on the page. OCR engines emit text-bearing regions as polygons — typically quadrilaterals — but the pipeline of this thesis approximates each polygon as an axis-aligned rectangle, represented by the image-pixel coordinates of its top-left corner (x_1, y_1) and its bottom-right corner (x_2, y_2), together with a centroid (c_x, c_y) at the geometric centre of the rectangle. The axis-aligned approximation is sufficient for supplement labels, on which text is rarely strongly rotated and the precision lost by ignoring small in-plane rotations does not affect the downstream graph reasoning. The bounding box, in this representation, is the only spatial information any downstream stage receives. Once OCR has run, the original image is not consulted by the geometric, classification, graph, or association stages.

Among the things a bounding box does not encode is the **reading order** of the document. Reading order is the sequence in which a human would consume the tokens — left-to-right within each row, top-to-bottom across rows, with column breaks honoured where columns exist. OCR engines do not always return tokens in reliable reading order; they may return them in detector order, in internal processing order, or in an approximate layout order that is not guaranteed to match the order a human reader would follow. On flat single-column documents the engine's output and the human reading order coincide closely, but on multi-column labels and on labels whose rows have any vertical curvature, they diverge: the engine's stream may interleave tokens from the left column with tokens from the right column, place column headers below the data they govern, or break a single visual row across two non-adjacent positions in the stream. Recovering reading order from the bounding-box stream is therefore a structural-inference problem rather than a transcription one, and the geometric mechanisms introduced in the rest of this section are the primitives the pipeline uses to solve it.

1.3.2 Columns, Rows, and Headers

A bounding box locates a single token, but the meaning of a supplement label is carried by groups of tokens arranged into rows and columns. Neither rows nor columns are explicitly marked in the OCR output; both must be recovered from the bounding-box stream by reasoning about the spatial relationships between tokens. The pipeline therefore treats columns and rows as **inferred structures** — derived from the bounding boxes rather than read off them — and the rest of §1.3 is concerned with the primitives this inference uses.

A **column** is a vertical band of the label in which a single semantic role is repeated down the page: a column of nutrient names, a column of per-100g quantities, a column of per-serving quantities, a column of percentage values. In the ideal case of a flat label, all tokens belonging to the same column share a near-constant c_x, and column membership is decided by simple ranges over c_x. In practice, the pipeline infers column membership approximately, since labels exhibit local skew and curvature that cause c_x to drift down a column. A **row** is the horizontal counterpart: a band in which one nutrient's full record — name, quantity, unit, context, and any auxiliary fields — is reported across the columns. Rows are locally horizontal rather than globally aligned: on flat labels they coincide with constant c_y ranges, but on curved or skewed labels their orientation drifts across the page. The inference of column and row membership under these distortions is the central topic of the geometric and graph stages of the pipeline.

A **header** is a special class of row whose tokens label the columns rather than carry data values. Most supplement labels have one header row, near the top of the table, that names each column with a context phrase such as `per 100g`, `per serving`, or `% NRV`. Some labels have two header rows — a primary header naming the columns and a secondary header giving units or percentage references. A header row differs from a data row in two ways: its tokens are typically classified as CONTEXT, UNIT, or as percentage-related tokens rather than as NUTRIENT or QUANTITY, and the column-role inference of the pipeline (introduced in §1.5) treats header tokens as the column's canonical descriptor rather than as nutrient records. Identifying which row of a label is the header is itself an inference task, and the absence of a unique header signal — no font weight, no separator line that survives OCR reliably — is what forces the pipeline to identify headers from the semantic labels of their constituent tokens rather than from any visual cue.

1.3.3 Geometric Distortions

The flat-label assumption that simplifies column and row inference rarely holds in practice. Supplement labels are printed on physical packaging — bottles, jars, sachets, blister packs — and a label photographed in commercial conditions exhibits one or more of the following geometric distortions.

Skew is a small global rotation of the label about its centre. A label that should sit at zero degrees instead sits at, say, three degrees clockwise, so that what was a horizontal row of tokens becomes a slightly inclined line. Under the pipeline's axis-aligned bounding-box representation, the boxes themselves remain axis-aligned, but the token centroids of a logical row no longer share a common c_y value — they lie along an inclined line rather than a horizontal one. The principal effect on the pipeline is that the constant-c_y assumption for row membership breaks, and row inference must compensate.

Rotation is the more general case in which the label is rotated by an arbitrary angle, possibly large. A photograph of a bottle taken from below or held sideways can produce a label rotated by ninety degrees or more. Rotation is, mathematically, a generalisation of skew, but is treated separately because the magnitude matters: small rotations can be absorbed by the row-inference mechanism, while large rotations require a preprocessing step that re-orients the image before OCR is run. The latter case is outside the scope of the pipeline of this thesis, which assumes that input images are roughly upright.

Curvature is a local rather than global distortion. A label printed on a cylindrical bottle, photographed from the front, exhibits a horizontal compression at the edges of the label that no global rotation can describe. Two consequences follow. First, rows in different vertical regions of the label, or even different segments of the same row, may have different local orientations: a row near the top of a curved label may be near-horizontal while a row near the bottom is inclined by several degrees, and a single long row that spans a curved region may itself bend across its width. Second, the relationship between c_x and column membership is no longer linear: a column may drift sideways across the page as it descends, so that the constant-c_x assumption for column membership breaks in addition to the constant-c_y assumption for rows.

A common preprocessing strategy in the document-IE literature is to estimate a single global skew angle and to rotate the image to remove it before OCR is run. This works well when the distortion is global. When the distortion is local — as on curved or perspective-warped labels — global deskewing cannot recover row alignment, because there is no single rotation angle that flattens the page. The pipeline of this thesis therefore does not rely on a deskewing preprocessor and instead handles geometric distortion through the row, column, and direction-inference primitives developed in §1.3.4 and §1.5. The cost of this choice is methodological complexity: the inference primitives must be robust to the distortions that a deskew step would otherwise have eliminated. The benefit is that the pipeline does not break on labels for which no global deskew exists — the curved and locally-warped cases that the dataset contains.

1.3.4 Context-Header Attachment Under Column Slope

A particular failure mode arises on multi-context labels when the columns themselves are not perfectly vertical. EU labels routinely report nutrients twice in adjacent columns — once per 100 g and once per serving, or once per 100 g and once per daily dose — and each data row contains a quantity in each column, both governed by their respective column header. Each data quantity must be attached to the correct header so that the predicted tuple records the correct context. On flat labels this attachment is straightforward: the data token at horizontal position c_x is attached to whichever header sits directly above it, where "directly above" means "smallest $|c_x^{\text{data}} - c_x^{\text{header}}|$". A simple horizontal-distance comparison is sufficient.

This breaks when the columns slope. Consider a multi-context label printed on a slightly skewed surface, or photographed at a small angle, so that the per-100g column drifts a few dozen pixels to the right as the table descends. At the top of the table, the per-100g header sits at $c_x = 200$ and the per-serving header at $c_x = 320$. At the bottom of the table, the per-100g column has drifted sideways and a per-100g data token now sits at $c_x = 280$. The horizontal distance from this token to its own header is $|280 - 200| = 80$ pixels, and the horizontal distance to the per-serving header is only $|280 - 320| = 40$ pixels — half as far. The naive horizontal-distance heuristic attaches the token to the per-serving header instead of the per-100g header, the predicted tuple records the wrong context, and the 4F F1 metric records the failure.

The fix is to attach data tokens to headers via the **direction the column itself runs in**, not via horizontal distance to the headers' positions at the top of the table. Each column's direction can be estimated from the line of best fit through the centroids of the tokens that have been preliminarily assigned to that column — for example, by an initial coarse c_x-based grouping, or

by restricting the fit to high-confidence column members whose membership is unambiguous from local geometry alone. Because columns on a supplement label are approximately vertical, the appropriate parameterisation is $x = a_k\, y + b_k$, in which y (the vertical position) is the independent variable and x (the horizontal position) is the predicted column position at that height. This form is numerically stable for nearly-vertical columns, where the alternative $y = m_k\, x + b_k$ form would have an unbounded slope. Given the fit, the perpendicular distance from a data token at position $p = (c_x^{\text{data}}, c_y^{\text{data}})$ to column k's line is

$$d_k(p) \;=\; \frac{\left|c_x^{\text{data}} - a_k\, c_y^{\text{data}} - b_k\right|}{\sqrt{1 + a_k^2}},$$

obtained by rewriting the line in implicit form and applying the standard point-to-line distance formula. The data token is attached to the column k — and therefore to column k's header — that minimises $d_k(p)$, rather than to the column whose header has the smallest horizontal-distance value $|c_x^{\text{data}} - c_x^{\text{header}}|$.

This formulation generalises the flat-label and slope-affected cases under a single mechanism. On perfectly flat labels, every column's best-fit line is vertical (slope $a_k = 0$ in the $x = a_k\, y + b_k$ form) and the perpendicular-distance criterion reduces to horizontal-distance matching, so no special-case code is needed. On skewed labels, each column's best-fit line is inclined and the perpendicular-distance criterion accounts for the slope, so a data token whose horizontal position has drifted sideways is still attached to its proper column. On curved labels, the linear fit is an approximation rather than an exact description of the column path; whether the approximation is sufficient to recover the correct attachment is an empirical question that the experimental chapter examines image by image. The mechanism is one specific instance of the broader pattern that the geometric stages of the pipeline implement: instead of measuring static distances in image-pixel coordinates, the pipeline first estimates the *local* coordinate frame of each structural unit (here, the column) and then measures distances in that frame.

1.4 Token Classification and Named Entity Recognition

1.4.1 The Classifier-Stage Design Space

The classification stage of the pipeline assigns a semantic category to each OCR token or token span after the post-OCR repair stage has finished. The categories are a small fixed set: NUTRIENT, QUANTITY, UNIT, CONTEXT, NOISE, and UNKNOWN. NUTRIENT names what is being measured; QUANTITY and UNIT carry the value and its unit; CONTEXT records the reference base under which the value is reported; NOISE marks tokens that are part of the label but are not part of any nutritional fact; UNKNOWN marks tokens that resist all the classifier's rules. Every downstream stage of the pipeline — graph construction, association, evaluation — depends on the classifier having assigned correct labels to as many tokens or spans as possible.

Three families of approaches to this kind of token classification exist in the document-IE and natural-language-processing literature, and the thesis evaluated all three as candidate implementations of the classification stage. The first is **rule-based classification**, in which the classifier is a hand-engineered set of lexicons, regular expressions, and decision rules that

the developer writes before any image is processed. The second is **zero-shot named-entity recognition** in the GLiNER style, in which a neural model trained on labelled NER data from other domains is asked at inference time to label spans on the supplement-label tokens, with the entity types specified as natural-language strings rather than as a fixed schema fitted on this data. The third is **embedding-based prototype matching**, in which a pre-trained multilingual sentence encoder produces vector representations of each candidate token, phrase, or normalised string, and compares them against vector representations of hand-written category prototypes by cosine similarity. The remainder of §1.4 introduces each family in turn, and §1.4.5 contrasts them along the dimension that matters most for this thesis: the kind of prior knowledge each one relies on to achieve its zero-shot capability.

1.4.2 Rule-Based Classification

The simplest of the three classifier families is the **rule-based classifier**, in which classification decisions are made by deterministic rules written by the developer from domain knowledge. The classifier consists of three kinds of components: **lexicons** that enumerate the canonical surface forms of each category (a list of nutrient names in the languages the labels appear in, a list of unit symbols, a list of context surface variants), **regular expressions** that match patterns the lexicons cannot enumerate (numeric quantities, percentage values, NRV strings), and **decision rules** that combine the lexicon and regex matches with positional or contextual cues to commit to a final label. In this implementation, rules are evaluated in a fixed priority order, and the first rule that fires determines the token's label.

The strengths of the rule-based approach are interpretability and the strong sense of zero-shot. Every classification decision is traceable to a specific rule that the developer wrote and can inspect; when a token is labelled incorrectly, the responsible rule can be identified, modified, or supplemented without retraining anything. The approach also satisfies the strictest reading of zero-shot: no learned parameters exist anywhere in the classifier, no labelled training data is used to fit parameters, and the only domain knowledge brought to bear is whatever the developer has written into the lexicons and rules. The classifier is also computationally inexpensive, since each token is processed by a small fixed sequence of lexicon lookups and regex tests.

Rule-based classification is also a natural fit for a pipeline whose downstream stages depend on the *stability* of category labels rather than on rich semantic generalisation. The graph constructor of §1.5.2 reasons over edge types defined by category labels, and the association stage of §1.5.4 propagates context through column-role inference. Both stages require that a token labelled NUTRIENT in one image be labelled NUTRIENT in another by the same logic, so that the graph operations are comparable across images. A rule-based classifier provides this stability by construction: identical inputs produce identical labels, with no run-to-run variability and no dependence on which images the classifier has been exposed to.

The limitations of the rule-based approach are also direct consequences of its construction. **Lexicon coverage** is the principal one: a token whose surface form is not in the lexicon is classified as UNKNOWN regardless of how clearly it represents a nutrient or unit to a human reader. New product categories, new vitamin shorthand, OCR-induced surface variants, and unusual translations all require manual lexicon extension. **Surface-form variation within**

categories is a related problem: the lexicon for nutrient names must enumerate German, English, French, Dutch, and other language forms of every nutrient the pipeline expects to see, and missing any one of them means the corresponding tokens fall through the rules. **Maintenance cost** is the third limitation: as the lexicon grows, the developer's mental model of which rules fire on which tokens grows in complexity, and adding new rules without disrupting existing behaviour requires careful regression testing on a representative held-out subset.

1.4.3 Learned NER as a Classifier Alternative

The second classifier family is **zero-shot learned named-entity recognition**. A neural model trained on labelled NER data from many domains is presented at inference time with a list of natural-language entity-type strings — for example, `nutrient name`, `unit of measurement`, `numeric quantity` — and asked to label spans of the input text as belonging to one of those types. The model has not been trained on supplement labels and has not been fine-tuned for the schema of this thesis; the *task* of mapping spans to user-supplied types is what the model has learned to perform, and the supplement-label schema is one instance of that task.

Modern zero-shot NER models are typically built on transformer encoders. The encoder consumes the input tokens and produces a contextual representation for each one — a vector that captures the token's meaning in the context of the surrounding tokens, rather than in isolation. Transformer-based contextual representations are what make zero-shot generalisation possible: a token whose surface form was never seen during training can still be classified correctly if the surrounding context resembles contexts the model encountered. Pre-trained transformer encoders such as BERT [9] and its multilingual descendants are the foundation of this capability, and the representation they produce is the input substrate on which span-classification heads operate.

The canonical zero-shot NER system relevant to this thesis is **GLiNER** [10], a compact generalist NER model that uses a bidirectional transformer to jointly represent input text and a list of natural-language entity-type prompts supplied at inference time. The model performs parallel entity extraction: candidate spans in the input text are scored against each prompted entity type, and a span is committed as belonging to a type when its score exceeds a threshold and the type is the argmax over candidate types. Because the entity types are specified as natural-language strings rather than as a fixed schema fitted on training data, GLiNER can be redirected to a new schema by changing the prompts rather than by retraining. The thesis evaluates GLiNER as one candidate replacement for the rule-based classifier of §1.4.2.

Zero-shot learned NER's strengths are the converses of rule-based classification's limitations. **Lexicon coverage is no longer a bottleneck**: a nutrient name not enumerated in any lexicon can still be classified correctly if its surrounding context resembles other nutrient names the underlying transformer has been exposed to during pre-training and NER fine-tuning. **Schema flexibility** is also stronger: a new entity type — say, an allergen warning that the rule-based classifier was never written to handle — can be added to GLiNER's prompt without code changes. The costs are the converse of the rule-based virtues: classification decisions are no longer traceable to a specific rule, and the model's behaviour on a new image is harder to predict than the rule-based classifier's. The maintenance burden shifts from lexicon engineering to prompt engineering and threshold tuning. A further cost specific to a graph-

based pipeline is that GLiNER, like most learned NER models, operates on a serialised text representation rather than on the OCR token stream with bounding boxes. Predicted spans must therefore be mapped back to the OCR tokens that produced them so that the downstream graph stage can continue to reason about geometry — a non-trivial alignment step that adds engineering surface area and introduces additional failure modes when the serialisation reorders tokens or merges adjacent spans across what were originally separate OCR regions.

1.4.4 Embedding-Based Classification (Brief Overview)

The third classifier family is **embedding-based prototype matching**. Each token, phrase, or normalised string from the input is mapped to a fixed-dimensional dense vector by a pre-trained multilingual sentence encoder. Each semantic category is represented not by a single prototype but by a *prototype pool* — a set of natural-language exemplars covering the surface forms the category contains: known nutrient names across the languages of interest for the NUTRIENT category, unit symbols and their variants for the UNIT category, context phrases such as `per 100g` and their multilingual equivalents for the CONTEXT category. Each member of the prototype pool is mapped to a dense vector by the same encoder, and the category's representation in vector space is the union of these vectors rather than a single prototype centroid. Classification is performed by cosine similarity: a token is assigned to the category some member of whose prototype pool it is closest to, provided the similarity exceeds a threshold and the closest pool member is more similar than any pool member of any other category by at least a margin.

The approach is zero-shot in a different sense than rule-based classification or zero-shot NER. The encoder has been pre-trained on diverse multilingual text but has not been trained on supplement labels or on the specific schema of this thesis; the prototype pools are written by the developer rather than learned; and the cosine-similarity decision rule is non-parametric with respect to fitted model weights — the threshold and margin are tuned but not fitted by gradient descent. The zero-shot capacity comes from the encoder's general-purpose semantic representations, which can place an unseen nutrient name closer to a member of the NUTRIENT pool than to any member of any other pool because the encoder has learned — during pre-training on general text — the lexical and semantic regularities the prototype embeddings exploit. The technical details of this approach, the threshold and margin parameters that govern the decision rule, and the trade-offs that its evaluation in this thesis revealed are the subject of §1.6.

1.4.5 Three Faces of Zero-Shot Classification

The three classifier families introduced in §1.4.2, §1.4.3, and §1.4.4 are all called zero-shot in the document-IE and NLP literature, but the word means three different things across them. The differences matter for understanding the design space, because each family makes a different bet about what kind of prior knowledge is most useful for this particular task.

The **rule-based classifier** is zero-shot because no parameters are fitted from labelled data. The prior knowledge it exploits is whatever the developer has written into the lexicons, regular expressions, and decision rules. This prior is **explicit** in the strongest sense: every classifica-

tion decision is the consequence of a rule that exists in source code and can be inspected, modified, or reasoned about by any reader of the codebase. The rule-based classifier's capacity is bounded by the breadth of the developer's domain knowledge and by the patience of the developer to enumerate it.

Zero-shot learned NER is zero-shot in a different sense: it has been trained on labelled NER data from many domains, but not on the schema of this thesis. The prior knowledge it exploits is **transferred** — the model's parameters encode statistical regularities of the entity-recognition task that are presumed to generalise across schemas, supplemented at inference time by the natural-language entity-type strings the user supplies. The classifier's capacity is bounded by how well its training distribution covers the inferential regularities the supplement-label task requires.

Embedding-based prototype matching is zero-shot in a third sense: a multilingual sentence encoder has been pre-trained on diverse general-purpose text, and its embeddings are used as a similarity space against developer-written prototype pools. The prior knowledge it exploits is **distributional** — the encoder has learned lexical and semantic regularities from a large pre-training corpus, and the cosine-similarity decision rule depends on those regularities being present in the embedding space the encoder produces. The classifier's capacity is bounded by what the encoder has learned to represent and by the developer's skill at writing prototypes that exploit those representations.

The three families therefore stake their generalisation on three different sources: hand-coded rules, cross-task statistical transfer, and pre-trained embedding geometry. None of these sources is intrinsically superior; each is appropriate to a different setting, and the empirical question of which one works best for the supplement-label domain is examined in the experimental chapter. What §1.4 establishes here is that the three approaches are not interchangeable variants of one technique. They embody different commitments about what counts as legitimate prior knowledge for a zero-shot system to bring to a new task, and the choice between them is therefore a methodological commitment as much as an engineering one.

1.5 Graph Representations of Documents

1.5.1 Graphs as a Document Representation

The pipeline of this thesis solves two problems: extraction and association. Extraction asks what each token means in isolation — is this token a nutrient name, a numeric value, a unit, or something else? Association asks how the tokens that have been extracted relate to one another — which nutrient does this quantity belong to, which unit accompanies which value, which header governs which column of data. The two problems have different shapes: extraction is *classificatory*, in that each token or token span is assigned a semantic category; association is *relational*, in that the meaning of any single relationship depends on the surrounding context of other relationships. The natural data structure for relational reasoning is the **graph**.

A graph in the document-IE setting consists of **nodes**, which are the OCR tokens after classification has assigned them a semantic role, and **edges**, which encode relationships between tokens. The pipeline of this thesis uses several edge types — for example, `SAME_ROW` between tokens in the same logical row, `SAME_COL` between tokens in the same column, and

CONTEXT_SCOPE between a header and the data tokens it governs — and the full set is enumerated in the methodology chapter. The graph is therefore not merely a description of the document's appearance but an explicit representation of its inferred structural relationships, with the geometric primitives of §1.3 providing the inputs from which most of the edges are constructed.

Recasting the association problem as a graph problem is what allows the pipeline to use the same machinery across very different layouts. A flat table, a multi-column panel, and a paragraph-style label all produce different distributions of node positions, but the graph operations that consume the resulting node-and-edge structures — pruning, matching, propagating context — are the same regardless of where the nodes came from. The remainder of §1.5 develops the specific operations these graphs support.

1.5.2 Edge Types and Soft Adjacency

Edges in a document graph carry meaning beyond mere connection between two nodes. Two tokens that share a SAME_ROW edge encode a different relationship than two tokens that share a SAME_COL edge or a CONTEXT_SCOPE edge, even when the underlying token pair is the same. The graph constructor therefore labels each edge with a **type** drawn from a fixed set, and the association stage that consumes the graph is sensitive to type: pruning operates differently on row edges than on column edges, and matching operates differently on context-scope edges than on either. Edge types are how a single graph carries multiple kinds of structural information without conflating them.

Each edge also carries a **weight** — a continuous scalar between 0 and 1 — that quantifies how strongly the relationship encoded by the edge applies to the specific token pair. The use of weighted rather than binary edges is a methodological choice related to the soft-adjacency idea used in graph-based document IE [11], in which adjacency is treated as a continuous-valued relationship between tokens rather than a binary one. PICK realises soft adjacency by *learning* the adjacency weights end-to-end from text, visual, and layout features. The pipeline of this thesis adapts the same idea to a deterministic, zero-shot setting: the weights are computed by closed-form functions of geometry rather than learned, but the underlying commitment — that the strength of a pairwise relationship varies continuously rather than binary — is preserved. The contrast that motivates this choice is between two ways of deciding whether two tokens are in the same row.

The simpler approach, **hard adjacency**, applies a binary distance threshold. Two tokens are in the same row if and only if the absolute vertical distance between their centroids is below some fixed value, say 25 pixels; otherwise they are not. Hard adjacency has the advantage of simplicity and the disadvantage of brittleness. A label printed in a slightly larger font has rows separated by 30 pixels and breaks the threshold; a label printed in a slightly smaller font has rows separated by 12 pixels and chains adjacent rows together. Worse, on a label whose row spacing is genuinely heterogeneous — header rows tall, data rows short, sub-nutrient rows tighter still — no single threshold value can be correct for all of them.

A weighted formulation instead assigns each candidate edge a weight that decays continuously with distance. Two tokens whose vertical centroid difference is small carry a high-weight SAME_ROW edge; two tokens whose difference is moderate carry a low-weight edge; two to-

kens whose difference is large carry no edge at all. The weight is computed by a deterministic function — typically a function that returns 1 at zero distance, decays smoothly as distance grows, and reaches 0 at some cutoff — that is calibrated against a *local scale* estimated from the surrounding neighbourhood, such as median token height or characteristic row spacing. The local-scale calibration is what allows the weight function to behave consistently across labels with different font sizes and different row spacings.

The pipeline of this thesis uses weighted edges of this kind for **spatial-distance edges** — SAME_ROW, SAME_COL, and others where the edge's existence and strength are functions of geometric proximity — because hard distance thresholds are exactly what break under the geometric distortions of §1.3.3. Other edge types, including CONTEXT_SCOPE, are **structurally inferred**: the edge endpoints are spatially grounded — a header is physically above the data tokens it governs — but the decision to create the edge is made by the column-role inference of the methodology stage, not by a distance computation. The weight of a structurally inferred edge either carries a fixed nominal value or reflects the confidence of the column-role inference itself. Weighted spatial adjacency and structurally inferred edges therefore coexist in the same graph, each appropriate to a different category of relationship.

1.5.3 Tail-Collision Avoidance

A weighted graph constructed from a dense supplement label routinely contains contradictions: edges that, if all retained, would violate the one-to-one structural constraints the document actually obeys. The most common contradiction is the **tail collision**: two nutrient nodes both connect, through high-weight SAME_ROW edges, to the same quantity node. The graph asserts that the quantity belongs to nutrient A *and* that it belongs to nutrient B. Both edges are individually plausible — both nutrients are spatially close enough to the quantity for the soft-adjacency function to assign each a non-trivial weight — but only one of them can be correct on a label that reports each nutrient with one quantity per context. The graph is internally inconsistent, and a downstream matching stage that consumes the graph naively will commit one of the two errors: assigning the quantity to the wrong nutrient, or splitting it ambiguously between both.

The technique that resolves this contradiction at the graph level, before matching, is **Tail-Collision Avoidance (TCA)**, introduced by SPADE [12] as a deterministic edge-resolution step in spatial-dependency parsing of semi-structured documents. SPADE motivates TCA from the observation that, in semi-structured documents, a node almost always has a single incoming edge per relation — with tabular structures being a notable exception. The original SPADE algorithm is iterative: at each tail node carrying multiple incoming edges, the highest-probability incoming edge is retained, the remaining edges are trimmed, the trimmed head nodes are redirected to their next-most-probable tails, and the process repeats until the graph becomes self-consistent or an iteration limit is reached. The pipeline of this thesis adopts a simplified, single-pass adaptation suited to its zero-shot setting: when two edges of the same type share a tail node, only the higher-weight edge is retained, and the lower-weight one is dropped, with no redirection step. This is sufficient for supplement-label tables, where the dense local geometry means that the trimmed head nodes are typically already covered by other high-weight edges from the surrounding row structure.

TCA is run after the soft-adjacency edges are constructed and before the bipartite matching stage of §1.5.4. The result of the simplified procedure is a graph in which each target node has at most one retained incoming edge of the relevant type. TCA is also type-sensitive: it operates on SAME_ROW edges and SAME_COL edges separately, because the one-to-one constraint applies within each type rather than across them. A nutrient may legitimately have a SAME_ROW edge to one quantity and a SAME_COL edge to a different one — that is how multi-context labels are structured — and TCA must not collapse such structures. The simplified TCA adaptation is purely a function of edge weights and edge types, and is therefore agnostic to whether those weights came from a learned graph module or from a deterministic geometric function.

1.5.4 Bipartite Matching, with Rank-Under-Context as Fallback

After tail-collision avoidance has resolved the within-type contradictions of §1.5.3, the graph still contains a question that no edge-pruning step can answer alone: which of the surviving high-weight edges constitute the final set of associations that become the output tuples? Different field associations have different structural shapes. Nutrient-to-quantity association is one-to-one *within a given context column* — each nutrient has at most one quantity in the per-100g column and at most one in the per-serving column. Quantity-to-unit association takes one of two structural shapes depending on the label's typography. When each quantity is printed adjacent to its own unit (`12 g`, `8 g`), the association is one-to-one within a row. When the unit is given once in a column header and inherited by every quantity beneath, the association is one-to-many propagation rather than one-to-one matching, structurally identical to the header-to-data context case. And header-to-data context association is itself one-to-many — a single `per_100g` header governs every quantity token in its column. The matching machinery must therefore recognise the structural shape of each association before choosing how to resolve it.

For the one-to-one associations, the natural formal framework is **bipartite matching**: two sets of nodes connected by weighted edges of a given type, where the task is to choose a subset of edges such that no node is incident to more than one chosen edge and the total weight of the subset is as large as possible. The canonical optimum-finding algorithm for this problem is the **Hungarian algorithm** [13], which finds the maximum-weight matching in polynomial time. The pipeline of this thesis uses a simpler **greedy bipartite-matching heuristic**, faithful to the matching algorithm of §1.7.3: edges are sorted by weight from highest to lowest, and each edge is committed to the matching if and only if neither of its endpoints has already been committed to an earlier edge. The procedure terminates when no remaining edge has positive weight or when one of the two node sets is exhausted. The greedy heuristic is suboptimal in pathological cases — it can commit early to a high-weight edge that prevents a globally better matching — and the gap to the Hungarian optimum is treated as an empirical trade-off rather than a proven small quantity. The simplicity of the greedy procedure is preserved consistently across the pipeline and the evaluator, both of which use the same algorithm.

For the one-to-many associations — header-to-data context propagation, and the header-inherited unit case described above — bipartite matching is not the right framework. Headers govern all the data tokens in their column simultaneously, so the matching is "every member

of column k receives the context (or unit) of column k's header" rather than "each data token chooses a single edge to commit". The pipeline handles these cases through structural propagation rather than through edge-by-edge matching: once column-role inference has identified each column's header, the header's value is propagated to every data token in the column as a property, not as a chosen edge.

Bipartite matching on geometric edges works well when the underlying geometry is reliable. On a flat label with clean OCR, the highest-weight SAME_ROW edge from each nutrient leads to its correct quantity, and greedy matching commits the correct pairs in order. The matching breaks when the geometry is unreliable: severely curved labels where the row inducer has under-merged or over-merged, OCR noise that has scrambled the centroid positions of some tokens, or layouts where the per-serving column drifts so far that the SAME_ROW edges link tokens that are visually adjacent but logically belong to different rows. In all of these cases, the geometric edges either cannot be constructed or carry weights that mislead the matching.

The pipeline therefore retains a **structural fallback** for cases where geometric matching fails: **rank-under-context**. The principle is to match tokens not by their spatial position but by their ordinal rank within their column, conditioned on the shared column context. Concretely, the nutrients in the nutrient column are sorted top-to-bottom and given ranks $1, 2, 3, \ldots$; the quantities in the per-100g column are sorted similarly and given the same ranks; the quantities in the per-serving column likewise. The rank-1 nutrient pairs with the rank-1 per-100g quantity and the rank-1 per-serving quantity; the rank-2 nutrient with the rank-2 quantities; and so on. Rank-under-context recovers tuples in cases where the per-100g and per-serving columns have drifted so far apart geometrically that no SAME_ROW edge can usefully cross the gap, where geometric matching alone would fail.

Rank-under-context is not safe to apply unconditionally. The rule depends on the column rank-lists being mutually compatible: each column must contain the same number of data entries as the others, in the same vertical order, with no row-skip caused by a missing OCR token or a sub-row absorbed into the wrong column. When these preconditions fail, rank-1 in one column and rank-1 in another no longer correspond to the same logical row, and rank-based matching produces tuples that look correct on per-column counts but are misaligned on the actual label. The pipeline therefore engages rank-under-context only when two conditions hold: the column-role inference of the methodology stage indicates that each column is well-defined and well-populated, and the rank lists across the relevant columns have compatible lengths. When either condition fails, the affected tokens are left unmatched rather than matched incorrectly, on the principle that a missing tuple is more recoverable than a wrong one.

The result of these design choices is a hybrid matcher: the greedy bipartite path handles the typical case where geometry is reliable, the structural propagation path handles the one-to-many associations that bipartite matching cannot cover, and the rank-under-context fallback handles the cases where geometric matching has failed but column structure is still recoverable. The three modes complement rather than compete, and each is engaged only when its preconditions are met.

1.5.5 Deterministic versus Learned Graph Reasoning

The graph-reasoning primitives of §1.5.1 through §1.5.4 — typed edges with continuous weights, tail-collision avoidance, bipartite matching, structural propagation, rank-under-context — are situated within a body of work on graph-based key-information extraction in the document-understanding literature, and the most directly relevant prior systems are PICK [11] and SPADE [12]. PICK formulates key-information extraction by combining graph learning and graph convolution over text segments, with edge weights produced from textual, visual, and layout features and learned end-to-end from labelled documents. SPADE formulates semi-structured document IE as spatial dependency parsing over OCR tokens and their 2D coordinates, predicting head–tail dependency edges and including tail-collision avoidance as part of the graph-decoding step. Both systems are supervised: they require labelled examples of the target schema and learn the relevant graph machinery from those examples. The pipeline of this thesis differs from both in that it constructs the graph deterministically from geometric and lexical primitives, without any labelled training data, and combines these primitives with the structural-propagation and rank-under-context mechanisms of §1.5.4 in a configuration that is specific to this thesis's design.

The trade-off between learned and deterministic graph reasoning is the same one that runs through every stage of the pipeline. Supervised graph learning can achieve stronger performance on the datasets it is trained for: a graph module trained end-to-end on labelled receipts or invoices learns weights that capture domain-specific patterns that are difficult to encode manually. The cost is that supervised systems may not generalise reliably across domains without adaptation or retraining, and that they require labelled data the supplement-label domain does not have. The thesis adopts the deterministic alternative: it borrows the *architectural ideas* — soft adjacency from PICK, tail-collision avoidance from SPADE, the broader principle that document IE benefits from typed-edge graph reasoning — and re-implements them as functions of geometry and semantic role rather than as learned parameters. The architectural commitments survive the adaptation; the supervised-learning machinery does not.

This adaptation is not a degradation of PICK or SPADE; it is a different point in the design space. The architectural ideas of soft adjacency and tail-collision avoidance are independent of *how* edge weights are produced, and the same graph operations that consume a learned adjacency matrix can also consume a deterministic one, provided the scores are comparable within the pruning or matching step. What changes is the calibration: a learned weight reflects the network's training experience, while a deterministic weight reflects explicit assumptions about the geometric regularities of the input domain. The two converge when those assumptions sufficiently capture the relevant geometry; they diverge when the input contains regularities the assumptions have not anticipated. The empirical question of which regime is preferable for the supplement-label domain — where the input is geometrically constrained but variable, and where labelled data is scarce — is examined in the experimental chapter.

1.6 Embeddings and Semantic Similarity

1.6.1 Embeddings: From Word Vectors to Multilingual Sentence Encoders

An **embedding** is a fixed-dimensional vector representation of a textual unit — a word, a phrase, a sentence, or a longer passage — produced by a model that has been trained such that useful semantic or distributional relationships are reflected in the vector space. The fundamental utility of embeddings is that the geometry of the vector space carries semantic information: two strings that mean similar things often land at similar positions, and the cosine of the angle between their vectors often becomes a usable proxy for how similar their meanings are. This is what allows downstream stages to make similarity-based decisions without enumerating explicit rules — the model's training has effectively compiled enormous amounts of distributional information about natural language into geometric relationships.

Early embedding models operated at the **word level**. Word2Vec [14] and GloVe [15] learned a single fixed vector for each word in a vocabulary, by either predicting words from their contexts or counting co-occurrences in a large corpus. These models established the basic property that a vector space could be made to reflect linguistic regularities, and the famous analogy $king - man + woman \approx queen$ is the canonical illustration that semantic relationships transfer to vector arithmetic. The limitation of word-level embeddings is the per-word fixed vector: a word with multiple senses receives a single representation that conflates them, and a word never seen during training has no embedding at all.

The dominant embedding paradigm of the modern era is **contextual**. Transformer-based models such as BERT [9] produce a different vector for each token *in the context* of the surrounding tokens — the same word in different contexts produces different embeddings, and an unseen word is represented by composing its subword pieces, each of which the model has been exposed to. Contextual embeddings inherit the geometric semantics of word embeddings but reduce their limitations: polysemy is handled implicitly, and the open-vocabulary problem is mitigated by the subword tokeniser. The cost is that a token's embedding now depends on its surrounding text, so embeddings cannot be precomputed and looked up; they must be computed by running the model on the input each time.

A further development relevant to this thesis is the **sentence encoder** — a model that produces a single fixed-dimensional vector for an entire sentence or short phrase, rather than one vector per token. Sentence-BERT [16] introduced the standard recipe: take a pre-trained transformer encoder, pool the per-token outputs into a single vector, and fine-tune the result on sentence-pair similarity tasks so that the pooled vectors are aligned with semantic similarity rather than with the encoder's original masked-language-modelling objective. The sentence encoder is the natural unit of representation for prototype-matching classification, where each prototype is a short natural-language phrase and each candidate is a token, span, or normalised string.

The final development this thesis builds on is the **multilingual sentence encoder**. A multilingual encoder is one that has been trained on text in many languages such that semantically equivalent strings in different languages land at nearby positions in the embedding space. This property is essential for supplement labels, where the same nutrient may appear in German on one product, English on another, and Italian on a third, and a classifier whose generalisation

depends on cross-lingual semantic alignment requires an encoder whose embedding space respects that alignment. The specific multilingual sentence encoder this thesis uses, BGE-M3, is the subject of the next subsection.

1.6.2 BGE-M3 and Cosine Similarity

The multilingual sentence encoder used in this thesis is **BGE-M3** [17], a publicly released model whose name reflects three design properties advertised by its authors: *multi-functionality* (it produces three kinds of representation — dense, sparse, and multi-vector — though the pipeline of this thesis uses only the dense representation), *multi-linguality* (it has been trained on text in over a hundred languages), and *multi-granularity* (it accepts inputs ranging from individual tokens to passages of several thousand tokens). For each input string, the model produces a single dense vector of fixed dimension that the pipeline uses as the input string's embedding.

BGE-M3 is the result of a self-knowledge-distillation training scheme that combines retrieval supervision in many languages with a unified loss across the three representation types. The practical consequence for this thesis is that the encoder produces dense vectors that are aligned across languages: a German string and its English equivalent tend to land near one another in the vector space, and the cosine similarity between them tends to be higher than between unrelated strings, which is the property the embedding-based classifier depends on. The prototype pools introduced in §1.4.4 contain exemplars in several languages, and a candidate token must be matchable against any of them regardless of the language it itself appears in.

The metric used to compare two embeddings produced by BGE-M3 in its dense mode (which, as noted above, is the only representation type the pipeline of this thesis uses) is **cosine similarity**. Given two vectors $\mathbf{u}$ and $\mathbf{v}$, the cosine similarity between them is defined as

$$\cos(\mathbf{u}, \mathbf{v}) \;=\; \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|\,\|\mathbf{v}\|},$$

the inner product of the vectors normalised by their lengths. The result is a scalar in the interval $[-1, 1]$, where 1 indicates identical direction in vector space, 0 indicates orthogonality, and -1 indicates opposite direction.

Cosine similarity is preferred over Euclidean distance for embedding comparison for two reasons. First, sentence encoders such as BGE-M3 are trained with similarity objectives that operate on direction in vector space rather than on magnitude, so the *angle* between two vectors is the quantity the model has been trained to make meaningful, while the *length* of either vector is a less reliable signal. Second, cosine similarity is invariant to vector scaling: multiplying $\mathbf{u}$ by any positive constant leaves $\cos(\mathbf{u}, \mathbf{v})$ unchanged, which means the comparison is robust to the per-input length variation that arises from inputs of different lengths or token counts. Euclidean distance possesses neither property, and embeddings compared by Euclidean distance often produce counter-intuitive rankings in which a longer vector dominates over a semantically closer but shorter one.

1.6.3 Prototype Matching, Thresholds, and Margins

The embedding-based classifier's role within the broader classifier of §1.4 is restricted to **nutrient detection**: deciding, for each candidate string in a position where a nutrient could

plausibly appear, whether the string represents a nutrient or not. The output is binary — either the candidate is committed as a NUTRIENT, or it is rejected and routed to UNKNOWN. The classifier does not identify *which* nutrient the candidate represents; that decision, when needed downstream, is performed by lexicon lookup against the canonical nutrient names. Other classification roles — UNIT, CONTEXT, QUANTITY, NOISE — remain rule-based even when the embedding classifier replaces the nutrient-detection step.

The classifier maintains a single prototype pool $\mathcal{P}$, written by the developer, containing natural-language exemplars of nutrients across the languages of interest, including common abbreviations and chemical synonyms. At classifier startup, every member of $\mathcal{P}$ is embedded by BGE-M3 once and the resulting vectors are cached, so that classification at inference time requires only the embedding of the candidate.

For a candidate string with embedding $\mathbf{u}$, the classifier computes the score

$$s(\mathbf{u}) \;=\; \max_{p \in \mathcal{P}} \cos(\mathbf{u}, \mathbf{u}_p),$$

the cosine similarity to the *closest* prototype in the pool. The classifier commits the candidate as NUTRIENT if and only if $s(\mathbf{u}) \geq \tau$, where τ is a threshold parameter. If $s(\mathbf{u}) < \tau$, the candidate is classified as UNKNOWN.

The threshold τ is the only parameter governing the decision. A high τ rejects candidates whose closest nutrient prototype is too dissimilar — typical of OCR garble, fragments of non-nutritional prose, and out-of-distribution inputs the encoder cannot interpret — and is the principal defence against false-positive nutrient detections. A low τ admits more candidates as nutrients, recovering some that lie outside the prototype pool's coverage at the cost of false positives. The trade-off is one-dimensional, which is what makes the threshold tuneable through a one-dimensional sweep rather than the two-dimensional grid that would be required for a margin-based discriminator.

The threshold is set by **grid search on a held-out validation subset** of the labelled data. A range of τ values is enumerated, the classifier is evaluated on the validation subset under each value, and the value that maximises 4F F1 (or 3F F1, depending on the experiment's objective) is selected as the operating point. The held-out validation subset is disjoint from the evaluation/test set used for final reporting in the experimental chapter, so threshold selection does not leak information from the evaluation data into the reported metrics. This procedure is the only point in the embedding-based classifier where labelled data is consulted: BGE-M3 was pre-trained without supplement-label data, the prototype pool is written by the developer, and the cosine-similarity decision rule is training-free with respect to fitted model weights. The threshold selection is, however, an explicit form of hyperparameter tuning on labelled data, which means the embedding-based classifier sits at a slightly weaker zero-shot point in the design space than the rule-based classifier of §1.4.2 — a methodological caveat acknowledged in §1.1's zero-shot commitment paragraph.

The simplicity of a one-dimensional threshold sweep is what makes the embedding-based classifier amenable to systematic ablation. The experimental chapter presents threshold sweeps that hold all other components of the pipeline fixed and vary only τ, allowing the resulting performance curve to be plotted explicitly. The shape of this curve — where the optimum lies, how broad the high-performing region is, how steeply performance falls off on either side — is

itself an empirical signal about how well the encoder's embedding space distinguishes nutrient inputs from non-nutrient inputs.

1.6.4 Failure Modes of Prototype Matching

The embedding-based classifier's performance depends on properties of the encoder's embedding space that are not under the developer's control. When those properties hold for a particular candidate string, the binary nutrient/non-nutrient decision is reliable; when they fail, the classification can be wrong in ways that no amount of threshold tuning can fully correct. This subsection names the principal failure modes that arise in practice on supplement-label data, both as a calibration of expectations for the experimental chapter and as context for the methodological choices §1.4 has already laid out.

The first failure mode is **out-of-distribution candidates rejected as UNKNOWN**. A candidate string that consists of OCR garble, partial words, or fragments of non-nutritional prose typically produces an embedding that is not close to any prototype in the pool. The maximum cosine similarity is below the threshold τ, the decision rule fails, and the candidate is routed to UNKNOWN. This is the *intended* behaviour: the threshold is a defence against forced commitments on inputs the encoder cannot interpret. But it is also a failure mode in the sense that some out-of-distribution candidates carry latent nutrient identity that a more aggressive classifier might recover. A token like `vit.B12*` after artefact stripping might genuinely refer to vitamin B12, but if the encoder embeds it far from the nutrient prototypes — because the truncated form and the asterisk artefact disrupt the encoder's expected input distribution — the classifier rejects it. Lowering τ to recover such cases comes at the cost of admitting genuinely non-nutrient inputs as false positives.

The second failure mode is **non-nutrient candidates falsely admitted as NUTRIENT**. Because the binary decision partitions the embedding space into a nutrient region (above the threshold) and a non-nutrient region (below it), any non-nutrient candidate that the encoder happens to embed close to a nutrient prototype will be misclassified as a nutrient. The most common cases on supplement labels involve tokens that lie in the same semantic neighbourhood as nutrients without being nutrients themselves: chemical names of non-nutrient compounds, regulatory or marketing claims that mention nutrients indirectly ("supports vitamin C absorption"), and prose fragments that cite a nutrient in a non-extractive context. Tightening τ to reject these false positives risks rejecting genuine nutrients whose embeddings are also moderately close to non-nutrient regions of the space — the trade-off is the threshold's central tension.

The third failure mode is **multilingual drift**. BGE-M3's cross-lingual alignment is a tendency rather than a guarantee. The drift can manifest in either direction. A genuine nutrient term in a language sparsely represented in the prototype pool may produce an embedding that is closer to a non-nutrient region than to the pool's nearest prototype, leading to a false rejection. Conversely, a non-nutrient term whose embedding the encoder has placed close to nutrient-region prototypes for cross-lingual reasons — perhaps because the term shares lexical or distributional patterns with nutrient names in another language — may be falsely admitted as a nutrient. This failure mode is hardest to diagnose and is the one most directly tied to the encoder's training distribution.

These three failure modes do not exhaust the ways binary nutrient prototype matching can fail, but they account for the kinds of misclassification that arise routinely on the supplement-label domain. The methodological response is twofold. First, the threshold sweep of §1.6.3 maps the operating-point range and identifies the value that best balances precision and recall in aggregate, accepting that no single threshold eliminates the failure modes individually. Second, the rule-based classifier of §1.4.2 remains the production choice precisely because its failure modes are different in kind — lexicon coverage gaps and surface-form variation rather than embedding-space pathologies — and the experimental chapter compares the two failure-mode profiles empirically to inform which approach is better suited to the supplement-label task.

1.7 Evaluation

1.7.1 Precision, Recall, and F1

Every metric reported in this thesis is built from three quantities: precision, recall, and F1. Let $\mathcal{P}$ be the set of predicted tuples produced by the pipeline on a test image, and let $\mathcal{G}$ be the set of ground-truth tuples annotated for that image. A *matching* $M \subseteq \mathcal{P} \times \mathcal{G}$ pairs predictions with ground-truth tuples under a chosen *match predicate* — a binary function that decides, for any (p, g) pair, whether p counts as a correct extraction of g. From the matched count $|M|$, the standard definitions follow:

$$P = \frac{|M|}{|\mathcal{P}|}, \quad R = \frac{|M|}{|\mathcal{G}|}, \quad F_1 = \frac{2\,P\,R}{P + R}.$$

Precision answers "of all the tuples the pipeline emitted, what fraction were correct?"; recall answers "of all the tuples the ground truth contains, what fraction did the pipeline find?"; F1 is the harmonic mean and penalises any imbalance between the two. What distinguishes one evaluation regime from another is not the formulas — these are fixed across the field — but the match predicate. The remainder of this section makes the predicates of this thesis explicit.

1.7.2 Token-, Span-, and Tuple-Level Predicates

Three families of match predicates are common in the document-IE literature, and they differ in how strictly they require a prediction and a ground-truth item to agree. A *token-level* predicate counts individual labelled tokens: a token is correct when its predicted label matches its ground-truth label, regardless of which entity that token belongs to or whether the surrounding entity span is correctly bounded. A *span-level* predicate counts correctly bounded entity spans: a span is correct when its boundaries and its label both match the ground truth, but the span is evaluated in isolation from any other span in the document. A *tuple-level* predicate counts structured records: a tuple is correct when *all* fields of the record match the ground truth simultaneously, and the predicted fields actually correspond to the same underlying fact on the page rather than being extracted from unrelated parts of the document. The three predicates form a strict hierarchy of difficulty — every span-level error is also a token-level error, and every tuple-level error is also at least one span-level error — but the converse does not hold, and gains at one level do not automatically translate to gains at another.

This thesis reports both token-level and tuple-level F1, but the headline metric is the tuple-level score. The reason for keeping both is that they answer different questions. Token-level F1 measures the classifier in isolation: it tells you whether the right tokens are being labelled with the right semantic categories, independently of how the pipeline later assembles them. Tuple-level F1 measures the pipeline as a whole: it tells you whether the final structured output is correct, which is the question a downstream consumer of the pipeline actually cares about. The two metrics can move in opposite directions. A more aggressive classifier can improve token-level F1 by labelling more tokens as nutrients, but if those extra tokens are noise that the graph constructor cannot place coherently, tuple-level F1 falls. Conversely, a stricter classifier that drops borderline tokens can hurt token-level F1 but produce cleaner tuples downstream. The thesis reports both so that any claim of the form "stage X improved" is anchored to the level at which the improvement was measured, and so that the experimental chapter can distinguish a real end-to-end gain from a localised improvement that did not survive to the output.

1.7.3 The 4-Field Tuple F1 of This Thesis

The headline metric of this thesis is the F1 of the 4-field tuple. A predicted tuple takes the form ⟨nutrient, quantity, unit, context⟩, and matches a ground-truth tuple if and only if all four fields agree under a normalised comparison. Three-field F1 (3F F1) — which drops the context field — is reported alongside as a secondary metric, both because the context field is the field most sensitive to OCR-driven column-header errors and because the contrast between 3F F1 and 4F F1 quantifies, image by image, how much performance is being lost specifically to context-attribution failures.

The match predicate operates field-wise. The nutrient comparison applies a normalisation that lowercases, strips diacritics, folds German umlauts using the standard transliteration convention (ß → ss, ä → ae, ö → oe, ü → ue), removes parenthetical qualifiers, and discards trailing punctuation; if the strings then agree, the field matches. The quantity comparison parses both predicted and ground-truth strings into numeric values — normalising the German decimal comma to a period and stripping the leading < from trace-amount expressions such as `<0.5` — and accepts the parsed values as equal only if they are identical. The unit comparison applies a synonym table that unifies common variants (μg, mcg, ug; kJ/kcal, kcal/kJ; ml, mL). The context comparison normalises whitespace and applies a translation table mapping surface variants ("PRO PORTION", "Je 100g", "per 100 g") onto a closed set of three canonical labels: `per_100g`, `per_serving`, and `per_daily_dose`.

The matching between $\mathcal{P}$ and $\mathcal{G}$ is one-to-one. Naively, a single predicted tuple could match multiple ground-truth tuples (when the label has duplicate nutrients in different contexts, for example), or multiple predictions could match the same ground-truth tuple. Both cases would inflate the matched count. The thesis follows the standard convention from the document-IE literature: an $N \times M$ score matrix is computed by evaluating the predicate on every (p, g) pair, the matrix is then traversed greedily in score-descending order, and at each step the highest-scoring unmatched pair is committed to the matching M and both rows are removed from further consideration. The procedure terminates when no unmatched pair scores above zero. This guarantees that $|M| \leq \min(|\mathcal{P}|, |\mathcal{G}|)$ and that the same prediction cannot be credited twice. The score function used to rank pairs in the matrix is the count of fields that agree

under the predicates above, with a small unit-agreement tiebreaker (`unit_bonus`) for pairs that tie on field count.

1.7.4 Normalisation and the Two-Tier Evaluator

The match predicates of §1.7.3 are deterministic: they apply fixed lookup tables, fold a small set of diacritics, and decide on string equality. They handle the bulk of the cases that arise in practice, which is why they constitute the *fast tier* of the evaluator. But they do not handle every case in which a predicted tuple and a ground-truth tuple denote the same nutritional fact. Two pairs of strings can describe the same nutrient and yet fail every deterministic rule: a chemical name and its common name (`Cobalamin` and `Vitamin B12`); two equivalent multilingual surface forms after umlaut folding has been applied to one but not the other; a compound nutrient name and its conventional shorthand (`saturated fatty acids` and `saturates` on UK and EU labels). A deterministic evaluator marks all of these as mismatches and underestimates the pipeline's true accuracy.

The thesis evaluator therefore adopts a two-tier design. Pairs that match deterministically are accepted by the fast tier and never escalated. Pairs that the fast tier rejects but that *plausibly* denote the same fact — by some heuristic such as token overlap, edit distance, or shared numeric content — are escalated to the *LLM tier*, in which a language model judge (Qwen-2.5:7b, served locally via Ollama in the reference implementation) is shown both tuples and asked to return a binary equivalence verdict. The LLM tier is bounded both in scope (only fast-tier rejects are escalated) and in cost (the LLM is queried only on borderline cases, not on every pair).

The cost of the LLM tier is reported alongside the headline F1 as a separate metric, `context_cost_pp` — the percentage-point increase in 4F F1 attributable to LLM-accepted matches. This is the maximum amount by which the headline F1 could be inflated if the LLM judge were biased toward over-acceptance. Reporting it separately lets the reader see, for every experiment, what the score would be under the fast tier alone and what the LLM tier added. For example, a predicted tuple ⟨"Vitamin B12", "2.5", "μg", `per_daily_dose`⟩ and a ground-truth tuple ⟨"Cobalamin", "2.5", "μg", `per_daily_dose`⟩ would be rejected by the fast tier on the nutrient field — the strings do not match after normalisation — but accepted by the LLM tier on the grounds that "Cobalamin" and "Vitamin B12" name the same molecule.

Two design choices about the LLM tier deserve naming. First, the LLM is used only as an evaluator and never as an extractor. The pipeline does not consult an LLM during inference; the LLM only enters at scoring time, and only on pairs the deterministic tier has already rejected. This isolation is what allows the cost of the LLM tier to be bounded and reported. Second, the LLM judge is a relatively small open-weight model running locally rather than a frontier API model. The choice favours reproducibility, cost, and data control over absolute judging accuracy, and it follows the convention in zero-shot evaluation that the judge should be at least one capability tier below the strongest available model so that the evaluation does not become an artefact of a single proprietary system.

1.7.5 Per-Image Variance and Diagnostic Logging

Aggregate F1 across the dataset is a single number, and a single number can hide the structure of the pipeline's failures. An aggregate 4F F1 of 41.5 percent on the supplement-label evaluation set could in principle reflect any number of underlying distributions, and on this dataset it reflects a wide spread: a substantial set of images on which the pipeline reaches 80 percent or higher, a larger majority on which it lands in the middle of the range, and a tail of images on which it produces nothing at all because the OCR is unrecoverable, the layout is unusually-arranged, or the structured path and the paragraph fallback both fail. The aggregate alone hides this spread. The thesis therefore reports per-image breakdowns alongside aggregate metrics, and the experimental chapter treats the per-image distribution as a first-class object of analysis — examining where the pipeline does well, where it fails, and why.

Four diagnostic CSV files are produced for every experiment. `evaluation_summary.csv` records the aggregate metrics — precision, recall, 3F F1, 4F F1, and `context_cost_pp` — together with the experiment name, the timestamp, and the configuration. `per_image_results.csv` records the same metrics broken out one row per image, allowing per-image comparison across experiments and ranking of best- and worst-performing images. `tuples.csv` records every predicted tuple in long format, allowing inspection of the raw output without re-running the pipeline. `pair_details.csv` records every (predicted, ground-truth) pair the matching algorithm considered, including which fields agreed under the predicates of §1.7.3, which is the file from which a per-field error analysis can be reconstructed.

Together, the four files are the audit trail that the modular-and-interpretable commitment of §1.1 promised. From an aggregate F1 number a reader can drill into the per-image distribution; from a regressed image they can drill into the predicted tuples; from a wrong tuple they can drill into the pair-level matching; and from there they can trace the failure backwards through the pipeline to the OCR token that produced it. Every claim in the experimental chapter is supported by at least one of these files, and the diagnostic logs are committed alongside the experimental results so that a reader who disputes a claim can reproduce the supporting evidence without re-running the pipeline.

1.8 Modular Pipeline Design Philosophy

The technical content of Chapter 1 has now been laid out across the preceding sections. Each one introduced a specific component or concept the pipeline depends on: OCR and post-OCR repair in §1.2, geometric primitives in §1.3, classifier families in §1.4, graph reasoning in §1.5, embedding-based prototype matching in §1.6, and the evaluation regime in §1.7, all anchored in the commitments stated at the outset in §1.1. This closing section steps back from individual components and articulates the architectural philosophy that runs through the chapter as a whole. The philosophy is not a new technique introduced here; it is the connective tissue between everything Chapter 1 has already covered, named explicitly so that the reader closes the chapter with a coherent mental model of how the pipeline is organised and why.

The first principle is **stage isolation through typed contracts**. The pipeline is organised as a sequence of stages — OCR and repair, classification and enrichment, graph construction, association, evaluation — and each stage has a fixed input schema and a fixed output schema

that constitute its contract with the surrounding pipeline. No stage reaches into another stage's internal state; no stage depends on the implementation choices of any other. The practical consequence is that single-component ablation experiments yield interpretable evidence: a stage can be replaced with a different implementation, the rest of the pipeline left untouched, and any change in pipeline output attributed unambiguously to the replaced stage. The alternative to typed contracts — a tightly coupled pipeline where stages share state — would compromise the interpretability of ablation results, because every replacement would risk silent ripple effects elsewhere in the pipeline.

The second principle is **diagnostic logging at every stage**. Beyond the data each stage emits to the next, each stage also emits a structured record of the decisions it made internally: which correction rules fired, which classifier rules were tried, which edges the graph constructor pruned, which pairs the matcher committed and at what score. The four CSV files introduced in §1.7.5 — `evaluation_summary.csv`, `per_image_results.csv`, `tuples.csv`, `pair_details.csv` — represent one externalised view of this logging, and together with the stage-level logs they aggregate, they form an audit trail in which every output tuple can be traced backwards to the OCR tokens that produced it, through the specific decisions that committed each link in the chain. The audit trail is what allows a regression on a single image to be localised to a specific stage, a specific token, and a specific decision, rather than being attributable only to "something changed somewhere".

The third principle is **deterministic implementations evaluated against learned alternatives**. Three of the chapter's sections name a deterministic-versus-learned trade-off explicitly: §1.4.5 (rule-based versus learned NER versus embedding-based prototype matching for classification), §1.5.5 (deterministic versus learned graph reasoning), and §1.6 (the BGE-M3-driven embedding classifier as a learned alternative to rule-based nutrient detection). At each of these decision points, the chapter has presented the deterministic and learned options as parallel candidates and identified the empirical comparison as a question the experimental chapter examines. The thesis does not claim in this chapter that one regime is preferable to the other in general; the choice depends on the specific domain, the available data, and the constraints the application operates under. What the chapter does establish is that the trade-off is real, that it appears at multiple stages of a complex pipeline, and that the design space at each stage admits more than one defensible choice.

Together, these three principles — stage isolation, diagnostic logging, and the deterministic-versus-learned framing — are what the pipeline of this thesis is designed to embody, and what the methodology and experimental chapters of the thesis will draw on as they describe the pipeline's specific implementations and report the empirical results those implementations produced.

2 Related Work

This chapter does the work Chapter 2 deferred. Chapter 2 introduced the technical primitives that the pipeline of this thesis depends on, restricting itself to the conceptual machinery the methodology and experimental chapters require, and leaving to the present chapter the question of how the thesis's design choices relate to the published systems that share its problem space. Chapter 3 takes up that deferred question. It surveys the research traditions the thesis sits within — visually rich document understanding, key information extraction from receipts and forms, zero-shot named-entity recognition, the application domain of nutrition-label extraction, and the methodologies for evaluating systems of this kind — and identifies the specific point in the design space the thesis occupies. Where a system has already been introduced in Chapter 2 — PICK, SPADE, GLiNER, BGE-M3 — the present discussion extends rather than duplicates the earlier treatment, focusing on the system-level positioning rather than the architectures Chapter 2 has already explained. Where a system is introduced for the first time — the LayoutLM family, Donut, Pix2Struct, the form-and-receipt benchmark lineage, the LLM-as-judge evaluation literature — the chapter supplies enough detail to make the comparison legible without restating common knowledge a reader of the document-understanding literature would already possess.

The chapter is organised as follows. §2.1 surveys the broader landscape of visually rich document understanding, contrasting the **cascaded-pipeline tradition** this thesis belongs to against the **end-to-end visual document understanding** paradigm that has come to dominate the document-AI literature, and explains why a pipeline approach remains the right choice for a labelled-data-scarce target domain. §2.2 narrows to table structure recognition specifically, treating it as an alternative paradigm whose extraction target differs from the thesis's. §2.3 covers OCR-free and vision-language document understanding as a second alternative paradigm that removes the staging the thesis retains. §2.4 returns to the graph-based key-information-extraction lineage that Chapter 2 borrowed from at the primitive level, and articulates the system-level positioning. §2.5 covers the zero-shot information-extraction research line on its own terms, distinguishing schema-flexible NER models from embedding-prototype matching and from LLM-prompted extraction. §2.6 discusses the application domain — nutrition labels and adjacent food-data extraction — and is candid about the relative scarcity of prior work on this specific target. §2.7 closes by stating where the thesis sits within the surveyed landscape.

2.1 Supervised document information extraction and benchmark-driven progress

2.1.1 The document-IE problem and its benchmark substrate

Document information extraction is the task of converting a document image, or a digital document with preserved layout, into a structured record of the facts the document contains. Unlike plain-text information extraction, the input carries spatial structure that is part of the meaning: a value placed inside a box on a form is the answer to the question printed beside the box; a number positioned in a table cell is the value of the column header above it and the row label to its left; an entry separated from the next by horizontal whitespace belongs to a different logical field. A document-IE system therefore consumes three signals — the textual content of the document, the geometry of where each piece of text sits, and, in many systems, the visual appearance of the document image itself — and produces some target structured representation: a tagged sequence of tokens, a set of typed entity spans, a list of key–value pairs, or a fully populated record schema.

The progress of the field over the last decade has been driven by a small number of benchmark datasets, each of which fixes a specific instantiation of the problem. **FUNSD** [18] is a canonical benchmark for form understanding: 199 noisy scanned forms with token-level annotations of four entity classes (header, question, answer, other) and explicit linking annotations between question and answer pairs. The benchmark rewards two related tasks — entity tagging and entity linking — and has become a standard test bed for the form-understanding subset of document IE. **SROIE** [19] is the scanned-receipt benchmark introduced at ICDAR 2019, with three tasks defined on the same underlying images: text localisation, OCR transcription, and key-information extraction over a four-field schema (company name, date, address, total). **CORD** [20] is a large receipt parsing dataset of Indonesian receipts introduced shortly afterwards, with multi-level semantic annotations: 30 fine-grained field labels organised into 8 super-classes, together with OCR boxes and reading-order annotations. Together these three datasets — forms, receipts at the field level, receipts at the parsing level — constitute a commonly used benchmark cluster for document KIE and receipt-and-form understanding, and the systems that have come to define the modern document-IE literature are in large part the systems that have done well on them.

A fourth dataset broadens the benchmark lineage in a different direction. **DocBank** [21] is a large-scale token-level annotated corpus of approximately five hundred thousand document pages drawn from arXiv submissions, with fine-grained layout annotations covering twelve component classes such as title, paragraph, list, table, figure, and equation. DocBank is not a KIE benchmark in the FUNSD/SROIE/CORD sense — it does not annotate typed key–value pairs or a target record schema — but a layout-analysis benchmark that asks systems to identify the structural role of each token in a document. Including it here is methodologically informative rather than directly comparative: while FUNSD, SROIE, and CORD represent KIE-oriented benchmarks, DocBank broadens the benchmark lineage toward token-level document layout analysis, and the systems that have done well on it tend to share architectural commitments with KIE systems in ways the next subsection draws out.

These benchmarks have made forms, receipts, and scientific documents central to document-

IE evaluation, but nutrition and supplement labels are not part of the canonical benchmark set, and §2.6 returns to that absence and what it has meant for the small body of nutrition-specific extraction work that exists outside the benchmark culture.

2.1.2 The supervised paradigm and what it assumes

What unites the systems that have come to define benchmark-driven progress in document IE is not a single architecture but a shared methodological regime. A model is first trained on large unlabelled document corpora using self-supervised objectives that exploit the document's text, layout, and often image content; the resulting parameters are then adapted to a labelled downstream dataset whose annotation schema defines the target output. The downstream dataset supplies both the *task* — entity tagging, key-value linking, full record extraction — and the *schema* against which performance is measured. This is the familiar pretraining-and-fine-tuning pattern, but in the document-IE setting it is made multimodal by the addition of layout coordinates and image evidence to the textual input that natural-language pretraining typically operates on. The regime's hallmark is that the schema the system is evaluated against is the schema for which labelled training examples have been provided, and the model's downstream performance depends on the availability and quality of those examples.

A central model family within this regime is the **LayoutLM lineage** [22, 23, 24] together with several closely related transformer-based document-understanding architectures, including **DocFormer** [25], **TILT** [26], and **BROS** [27]. Most of these systems combine textual, layout, and visual evidence into a single transformer encoder pretrained on large-scale unlabelled document corpora; others, such as BROS, show that strong KIE performance can also be obtained by concentrating on text and two-dimensional layout alone, with carefully designed spatial encodings standing in for the image-stream signal the multimodal models exploit. A representative example is the masked visual-language modelling objective used in the LayoutLM family and extended in LayoutLMv2 [23], where selected text tokens are masked and the model must recover them from surrounding textual, spatial, and visual context; LayoutLMv2 further adds text–image alignment and text–image matching objectives to strengthen cross-modal interaction during pretraining. After pretraining, the same encoder is fine-tuned on FUNSD-style entity-linking, SROIE-style or CORD-style KIE, or other downstream targets, with task-specific heads attached to the shared backbone.

The strength of this regime is also its condition. The benefits are real: end-to-end optimisation against a benchmark-defined objective tends to produce systems that are well-tuned to that objective; multimodal pretraining permits joint reasoning over modalities that would be hard to integrate by composing separate modules; and the leaderboard culture surrounding the benchmarks of §2.1.1 has produced a steady accumulation of architectural and training-recipe improvements that compound over successive model generations. The condition is that the regime presupposes labelled examples in the target schema. Adapting one of these systems to a new schema for which no labelled corpus exists requires either annotating one — which the authors of FUNSD, SROIE, CORD, and DocBank did at substantial cost — or some indirect path through cross-task transfer or zero-shot evaluation that the regime as such does not natively provide. The regime also depends on schema stability between annotation, fine-tuning, and deployment: if the target schema changes, labelled data and task heads often have to be re-

vised. These are conditions under which the regime's strengths are realised, and they are what §2.1.3 takes up next.

2.1.3 Where this thesis sits relative to the supervised regime

The regime described in §2.1.2 presupposes labelled examples in the target schema. This thesis works on a domain — EU supplement nutrition labels — for which no canonical labelled benchmark in the FUNSD, SROIE, CORD, or DocBank lineage exists, and is committed to operating zero-shot apart from the limited validation-based threshold tuning described in §1.1. The consequence is direct: applying the supervised regime to this thesis's setting would first require constructing the labelled corpus the thesis is committed to not presupposing, which would contradict the zero-shot commitment that motivates the architectural choices the rest of the thesis describes. The work of the remaining sections is therefore to examine research lines that either relax the labelled-data requirement, change the extraction target, or offer representations that can be re-implemented without supervised training.

The neighbouring research lines fall into three structural responses to this mismatch. The first is to *change the extraction target* by routing the problem through an architectural paradigm whose output is not the same record schema that supervised KIE consumes: **table structure recognition** (§2.2) reconstructs the cell grid of a tabular region as an intermediate representation from which records can be derived downstream, and **OCR-free vision-language document understanding** (§2.3) folds OCR and IE together into a single end-to-end model that emits the structured output directly from the document image. The second is to *retain the representation but re-implement it without supervised training*: **graph-based key information extraction** (§2.4), which is the lineage §1.5 has already drawn from, supplies the typed-edge document representation that the thesis's association stage uses, with the deterministic-versus-supervised choice §1.5.5 named at the level of individual primitives now revisited at the level of complete systems. The third is to *operate without labelled target-schema data in the first place*: **zero-shot information extraction and semantic matching** (§2.5) covers the schema-flexible NER lineage, prototype-based matching, and LLM-prompted extraction, all of which produce structured output by means other than fitting parameters to a labelled training set. The domain-specific check that completes the survey — whether the supplement-label setting has been previously addressed at all, and how the small body of nutrition-label-extraction work that exists relates to the canonical benchmark culture — is the work of §2.6.

The constraint set that defines what this thesis needs from these research lines is the one introduced in §1.1: the system should operate without target-schema training data, remain flexible across the semi-structured layouts found in supplement labels, and preserve enough modularity for errors to be inspected at the OCR, correction, classification, and association stages. The rest of this chapter therefore examines how neighbouring research lines — table-structure recognition, OCR-free document VLMs, graph-based KIE, zero-shot extraction, and nutrition-label parsing — meet, evade, or leave unresolved the constraints that define the supplement-label setting.

2.2 Table structure recognition

2.2.1 The table structure recognition problem and its representations

Table structure recognition (TSR) is the task of taking a region known or believed to contain a table and producing a structured representation of the table's grid. The input is the table image, optionally accompanied by OCR-recognised text and bounding boxes; the output is a description of the table's structural composition, including the positions and adjacency of cells, their membership in rows and columns, the relations between header and data cells, and any cell spans that cross the basic row–column grid. TSR is distinct from the two problems Chapter 2 has already covered. OCR maps pixels to text; KIE maps text and layout to typed records; TSR maps a table region to a grid representation that sits between the two and commits to neither typed semantics nor raw transcription as its target output.

The representations TSR systems emit fall into a small number of families. HTML-like markup is one widely used target representation, in part because PubTabNet [28] formulates image-based table recognition as the conversion of table images into corresponding HTML structure expressed through `<tr>`, `<td>`, and `<th>` tags with `rowspan` and `colspan` attributes for cell spans. A second representation is the logical–physical structure split, in which logical structure — which cell belongs to which row and column, which cells are headers, where spans occur — is separated from physical structure, the bounding-box coordinates of each cell in the image. TSR systems differ in whether they emit logical structure, physical structure, or both, and the two are sometimes produced by separate sub-modules within a single system. What unites these representations is what they describe: they describe the grid of the table — its cells, its rows, its columns, its headers, its spans — not the domain meaning of the cells beyond marking some of them as headers. The cells contain text after OCR has been applied, but TSR commits to no further typing of that text into application-level categories. This distinction is what the positioning argument in §2.2.3 turns on.

TSR is evaluated by metrics that reflect the same grid-versus-meaning distinction. TEDS [28] — Tree-Edit-Distance-based Similarity — evaluates the similarity between predicted and reference table structures through tree-edit distance over HTML-like table representations, computing how many insertions, deletions, and substitutions of nodes are required to transform the predicted markup into the reference markup. GriTS [29] — Grid Table Similarity — evaluates a predicted table more directly as a grid or matrix of cells, comparing predicted and reference structures in the natural matrix representation rather than only through markup edit distance, and can be applied to cell topology, cell location, and cell content recognition within the same grid-based framework. Both metrics are TSR-internal: they evaluate the recovered grid, not whether the cells contain the right typed records under any application-level schema.

2.2.2 Representative TSR systems and their architectural commitments

The architectural surface of TSR can be organised along the axis of how a system produces its grid representation. *Separator-and-boundary-based* TSR predicts row separators, column separators, or cell boundaries on the image, and assembles cells from the predicted lines or boxes. *Sequence-generation-based* TSR emits the table structure as a structured token sequence — typically as HTML or as an HTML-like markup — produced by a transformer decoder over

the table-image features. A third family, *hybrid systems*, predicts logical structure and physical cell locations jointly, often through dual decoding paths that share an encoder. The choice between these families has consequences for how the system handles spanning cells, irregular grids, and dense-text content, and the canonical TSR systems sit at different points on this axis depending on the trade-offs their authors prioritised.

Two systems are representative for the purposes of this section. **TableFormer** [30] uses a transformer-based encoder–decoder design in which one decoding path predicts the table structure as an HTML-like token sequence while a cell-bounding-box decoder predicts the physical location of table cells, jointly targeting logical structure and physical cell localisation in a single supervised end-to-end model. **SLANet** represents the lightweight production-oriented line of TSR systems in the PaddleOCR ecosystem; it predicts table structure and cell locations through a structure–location-alignment design rather than through separator-based assembly, with a PP-LCNet backbone, CSP-PAN feature fusion, and an SLA head that aligns structure and position information. SLANet is one of several modules integrated into PP-StructureV3 [31], the hierarchical document-parsing pipeline distributed alongside PP-OCRv5 in the PaddleOCR 3.0 release, with later SLANeXt variants providing dedicated weights for wired and wireless table recognition [32]. This point is especially relevant because the OCR engine used in this thesis comes from the PaddleOCR ecosystem, whose document-parsing stack includes PP-StructureV3 and the table-recognition modules just described. The thesis therefore does not ignore TSR because it was unavailable within the chosen ecosystem; it omits it because invoking TSR would add an explicit grid-reconstruction stage to the pipeline, and §2.2.3 sets out the specific reasons that addition was not made.

The benchmark substrate commonly used for TSR training and evaluation includes three datasets that are especially relevant here. **PubTabNet** [28] contains approximately 568,000 table images extracted from scientific articles, annotated with corresponding HTML structure, and has become one of the central benchmarks for image-based table recognition. **FinTabNet** [33] provides cell-level annotations for tables drawn from financial documents. **TableBank** [34] is a weakly supervised benchmark built by extracting tables from Word and LaTeX documents, containing approximately 417,000 labelled tables, with structure derived from the source markup rather than from human annotation. These three benchmarks have shaped the evaluation culture of TSR over the last several years and define what "good TSR performance" means in the published literature. TSR systems trained and evaluated on these benchmarks are therefore optimised for table distributions shaped by scientific, financial, or typeset document conventions, rather than by the photographed, curved, multilingual, and semi-tabular layouts that supplement labels often exhibit.

2.2.3 Why this thesis does not include a TSR stage

The exclusion of a dedicated TSR stage is a deliberate architectural choice rather than an omission. Three reasons motivate it: TSR's output schema does not match the thesis's target records; state-of-the-art TSR systems depend on supervised table-structure training data drawn from distributions unlike supplement labels; and a TSR-first pipeline would complicate layout handling by requiring routing between tabular and non-tabular cases.

The first reason is schema mismatch. As §2.2.1 established, TSR produces a grid represen-

tation: cells, rows, columns, headers, spans. The thesis produces typed records of the form ⟨nutrient, quantity, unit, context⟩, in which each field is assigned an application-level semantic role. Even a perfect TSR system would leave the typing problem — which cell is the nutrient name, which is the quantity — to a downstream stage that TSR systems are not themselves evaluated on. A TSR-first pipeline could still be built, but it would be a two-stage system: grid reconstruction followed by semantic interpretation of the recovered rows, columns, and headers. The thesis chose to do that semantic interpretation directly on the OCR token stream by means of the geometric and graph-based machinery introduced in Chapter 2, rather than materialise an intermediate grid representation that would still need to be typed before the target records emerged.

The second reason is supervision dependency. The TSR systems surveyed in §2.2.2 are supervised: they are trained on PubTabNet, FinTabNet, TableBank, and similar benchmarks, and their performance depends on the availability and distributional alignment of labelled table-structure training data. The closing observation of §2.2.2 already established that those benchmarks represent table distributions shaped by scientific, financial, or typeset document conventions, while supplement labels are photographed, often curved, multilingual, and frequently semi-tabular. Evaluating the domain transfer of supervised TSR systems to supplement-label distributions would itself involve substantial empirical work outside the scope of this thesis, and using supervised TSR would in any case reintroduce the labelled-data dependency that the zero-shot commitment of §1.1 is designed to avoid. The thesis's design constraint is therefore to find primitives that operate without supervised target-domain data, which the geometric and graph-based reasoning of Chapter 2 supplies and which the supervised TSR systems of §2.2.2 do not.

The third reason is layout-flexibility scope. Many supplement labels in the dataset are not arranged as clean fully-bordered grids: the corpus includes paragraph-prose nutrient declarations, single-column layouts, and multilingual slash-rows with values flowing alongside their multilingual nutrient names. A TSR-first pipeline would therefore need a layout-routing stage that decides whether a given label should be sent through TSR or handled by a non-tabular fallback path, and the fallback path would still need geometric reasoning over the OCR token stream of the kind §1.5 describes. The thesis chose to treat tabular and non-tabular cases under a single unified geometric-graph framework rather than route them through different architectural paths. In the terms of §2.1.3, TSR is therefore an alternative that *changes the extraction target*: it reconstructs a table before information extraction can begin. This thesis instead keeps the extraction target fixed as nutrient–quantity–unit–context association and treats tabular and non-tabular label layouts under a single geometric-graph representation.

2.3 OCR-free and vision-language document understanding

2.3.1 From OCR-dependent pipelines to OCR-free document models

The default architectural pattern for visually rich document IE has for some years been a cascade of three stages. An optical character recogniser produces text together with bounding boxes from the document image; a layout-aware encoder consumes the OCR output together with geometric features and, in many systems, visual features extracted from the same image;

and a task-specific head emits the target structured output. The cascade is what allows the textual, spatial, and visual signals introduced in §2.1.1 to be brought together for downstream reasoning, and it is the architectural pattern within which the supervised regime of §2.1.2 has accumulated its successive model generations. This is also the architectural family to which the pipeline developed in Chapter 2 belongs: OCR produces the token stream, post-OCR correction repairs it, and the later stages reason over the corrected tokens and their geometry rather than over the original image directly.

The cascaded pattern incurs three costs. The first, already identified in §1.2.1, is **OCR error propagation**: errors at the perceptual stage are unrecoverable by any downstream component, and a misread character or a missing token simply does not appear in the output. The second is **staging overhead**. The cascade requires a separate OCR engine, a post-OCR correction stage to repair its characteristic error distribution, and an interface between the OCR output schema and whatever the next stage expects, each of which adds engineering surface area and creates an interface at which assumptions between modules must be made explicit. A further cost is **information loss at the OCR boundary**: once the image has been converted into recognised text and bounding boxes, downstream stages no longer have direct access to visual cues such as font weight, colour, texture, or local image evidence that might inform classification or association decisions. These costs were accepted as the necessary price of OCR-based pipelines until multimodal pretraining at scale produced encoders that could attend to image patches directly without going through a recognised-text intermediate.

The OCR-free alternative reverses the architectural commitment. Rather than recover text from the image and reason over the recovered text, an OCR-free document model consumes the image directly through a vision encoder and emits the structured target — a tag sequence, a JSON-like record, a markup string — through an autoregressive decoder, without committing to a separate OCR stage at all. The shift is enabled by three convergent developments: image-to-text encoder–decoder architectures from the broader vision-language literature, large-scale multimodal pretraining on document and screenshot data, and structured-output decoding objectives that bypass the intermediate text representation. The point is not that OCR-free models contain no text-like internal representations; it is that the pipeline no longer materialises OCR text as a separate, intermediate artefact that downstream components consume. The named systems that instantiate these design choices are the subject of §2.3.2.

2.3.2 Representative OCR-free and vision-language systems

The canonical OCR-free document understanding system is **Donut** [35], one of the clearest early examples of the paradigm in practice. Donut combines a Swin-Transformer image encoder over the document image with a BART-style autoregressive decoder that emits the structured target as a sequence of tokens, including special tags marking field boundaries within the output. The model has no OCR sub-module: the encoder consumes image patches directly and the decoder generates structured text without any intermediate recognised-text representation being materialised. The pretraining recipe uses synthetically generated document images and their token sequences, followed by task-specific fine-tuning on the downstream IE schema. Donut's authors frame the model explicitly as a response to the costs §2.3.1 named: OCR cost, OCR inflexibility across languages and document types, and OCR error propagation. The sys-

tem's contribution is to demonstrate that document IE can be performed end-to-end without an explicit OCR stage, given sufficient pretraining and supervised adaptation to the target schema.

A second representative system, **Pix2Struct** [36], extends the image-to-text view beyond document forms by treating visually situated language understanding as a general image-to-sequence problem. Pix2Struct combines a vision-transformer encoder with a text decoder, but its distinguishing commitment is its pretraining objective: it learns to parse masked web screenshots into simplified HTML, training the model to map visual layout to structured text on a corpus drawn from the open web rather than from document-specific annotation. The pretraining task — predicting the missing portions of a page's HTML from a partially masked screenshot — produces a model whose representations are aligned with the structural conventions HTML encodes, and whose downstream behaviour on document IE, web UI understanding, infographic question answering, and related tasks is driven by that alignment rather than by document-specific inductive biases. Pix2Struct represents a different point in the OCR-free design space than Donut: the architectural shape is similar, but the pretraining commits the model to a more general image-to-structured-text capacity.

LayoutLMv3 [24] is relevant here as a contrast rather than as an OCR-free system. It remains OCR-dependent, because it consumes recognised words together with layout coordinates and image patches as input, but it is mentioned in this section as a bridge: its unified text–image masking and word–patch alignment pretraining objectives show how document AI moved from text-and-layout encoders toward more tightly coupled visual-language pretraining, and the architectural commitments it foregrounded — image patches as first-class input, cross-modal alignment as a pretraining objective — are part of what made the OCR-free transition more natural. LayoutLMv3 itself is in the supervised regime of §2.1.2 and consumes OCR output, but the design choices it made are visible in the OCR-free systems that followed.

A more recent use pattern is to prompt general-purpose document-capable vision-language models — such as the **Qwen-VL** [37] and **InternVL** [38] families, or proprietary systems such as Gemini and GPT-4V/GPT-4o-style models — to read a document image and emit JSON-like records in response to a natural-language instruction. In that use pattern, the model is not a document-specific OCR-free architecture in the Donut sense; the end-to-end extraction behaviour emerges from large-scale multimodal pretraining and instruction following, and the same model that handles a supplement label can be redirected to a receipt, a form, or a screenshot by changing the prompt. This is not the same architectural claim as Donut's OCR-free document transformer: it is an inference-time use pattern built on general multimodal instruction following. The distinction between specialised OCR-free document models and frontier VLMs used as extractors is what §2.3.3 sharpens into two separate positioning arguments.

2.3.3 Why this thesis retains explicit OCR and modular extraction

The choice not to adopt either of the alternatives surveyed in §2.3.2 — specialised OCR-free document models or frontier vision-language models used as extractors — is a deliberate architectural decision rather than an omission. Three reasons motivate it: schema-specific reliability and zero-shot control, modularity and interpretability, and deployment and reproducibility. The reasons split unevenly across the two §2.3.2 categories. Schema-specific reliability is primarily a concern about specialised OCR-free models such as Donut and Pix2Struct, which re-

quire task-specific adaptation to produce reliable schema-conformant output. Deployment and reproducibility are primarily concerns about the frontier-VLM-as-extractor use pattern, which carries version, cost, privacy, and auditability considerations the deterministic pipeline avoids. Modularity and interpretability apply to both categories, because the end-to-end generation that defines either alternative hides the intermediate decisions a modular pipeline exposes.

The first reason concerns schema-specific reliability and zero-shot control. For specialised OCR-free models, the problem is that the systems surveyed in §2.3.2 are pretrained on synthetic documents or web screenshots and require task-specific fine-tuning to produce reliable output in the target schema, which would reintroduce the labelled-data dependency that the zero-shot commitment of §1.1 is designed to avoid. The thesis would have to construct a labelled supplement-label corpus to fine-tune Donut or Pix2Struct against, which is precisely what the zero-shot commitment is designed to avoid. For frontier VLMs used as extractors, the situation is different: zero-shot prompting is available, but the resulting extraction behaviour exhibits its own control concerns, including variation in schema adherence across runs, model versions, and prompt formulations. The thesis's deterministic pipeline produces identical output for identical input by construction, which is a form of control that prompt-based extraction does not provide and that would itself become part of any maintenance regime built around it.

The second reason concerns modularity and interpretability, the third commitment of §1.1. End-to-end models — whether specialised OCR-free systems or frontier VLMs prompted at inference — emit a structured output as a single autoregressive generation, and the perceptual, correction, classification, and association decisions that produced any given output are not separately observable. The four diagnostic CSVs §1.7.5 names — `evaluation_summary.csv`, `per_image_results.csv`, `tuples.csv`, and `pair_details.csv` — and the stage-level audit trail they constitute would have no direct analogue in an end-to-end system. A failure on a particular tuple cannot be traced to a specific OCR token, a specific correction rule, a specific classification decision, or a specific graph-association edge. The thesis chose modularity because that traceability is part of what makes single-variable ablations a viable methodology and what makes the experimental chapter's stage-level causal claims defensible.

The third reason concerns deployment and reproducibility, primarily for the frontier-VLM-as-extractor use pattern. Frontier vision-language models are deployed as services whose model versions are updated by their providers, so a result obtained against one snapshot may not reproduce against the next; per-image inference cost and latency scale unfavourably for production document IE relative to a local pipeline; sending document images to a third-party API introduces privacy and data-control concerns that a deployer would have to address; and the resulting extraction behaviour cannot be audited by any party other than the model provider. None of these are objections to vision-language models as a research direction; they are deployment considerations specific to the use pattern in which a frontier VLM serves as the extractor, and they are part of why the thesis's pipeline operates locally on open-weight components and deterministic logic.

In the terms of §2.1.3, OCR-free document models and frontier-VLM-based extraction are alternatives that remove the staging that makes modular extraction possible. The thesis instead retains the staging, accepting the costs §2.3.1 named — OCR error propagation, staging overhead, and information loss at the OCR boundary — as the price of the diagnostic transparency and audit trail the modular pipeline provides.

2.4 Graph-based key information extraction

2.4.1 Graph-based KIE as a research line

§1.5.1 introduced graphs as a representation for document reasoning; this section situates that representation within the broader graph-based KIE literature. The framing of document IE as a *relational* problem follows from the observation that serialised text representations alone do not capture the structural relationships that determine meaning in visually rich documents. A document encodes key–value pairs in which the key and the value may be on the same row or in adjacent regions; header–data attachments in which a column or section header governs the data tokens beneath it; and row–column groupings in which membership in a logical row is determined by horizontal alignment rather than reading-order proximity. Tokens that are visually adjacent or structurally related may be far apart in any natural reading order, and conversely tokens close in reading order may belong to unrelated parts of the document. The natural data structure for representing relational reasoning of this kind is the graph: nodes correspond to tokens or token spans after OCR has been applied, and edges encode pairwise relationships between them. Graph-based KIE is the research line that has organised itself around this representation.

The systems that constitute this research line share three architectural commitments. First, they take OCR tokens or token spans as the nodes of a document graph constructed at inference time. Second, they encode pairwise relationships through edges that may be *typed*, weighted, or otherwise conditioned by the relation they represent — spatial proximity, alignment along a row or column, predicted dependency, key–value linkage — rather than treating all token-pair relationships as a single homogeneous adjacency. Third, they consume the resulting graph structure through graph-aware reasoning machinery: graph convolution, message passing across edges, or attention mechanisms that condition on edge information as well as on node features. Within these shared commitments, the systems differ on three axes that §2.4.2 will use to compare them: how the edges are produced — by a learned module trained on labelled examples or by deterministic rules over geometry and semantic role; how the graph is used downstream — as input to a graph neural network whose output is a tagged sequence, or as a structure consumed by a matching procedure; and what supervision signal is required for training, including whether labelled target-schema examples are needed at all. This final axis is the one most important for this thesis: some systems learn edges or graph-aware representations from labelled documents, whereas this thesis constructs its graph deterministically from geometry, semantic labels, and domain constraints.

2.4.2 Representative graph-based KIE systems: PICK, SPADE, and FormNet

PICK [11] is a supervised graph-learning system for key-information extraction from documents. The model takes OCR-recognised text segments together with their textual content, visual segment features, and 2D coordinates as input. Over these inputs, it learns a pairwise edge structure — an adjacency representation in which the weight associated with each token-pair edge conditions on textual, visual, and layout features jointly — so that the graph reflects relationships the model has been trained to find informative on labelled documents. The resulting weighted graph is then consumed by a graph convolutional network whose layers propa-

gate information across edges and produce node-level contextual representations for each text segment. A downstream tagging head emits the key-information predictions, and the entire pipeline — feature encoders, edge-weight learning, graph convolution, and tagging head — is trained jointly end-to-end on labelled examples in the target schema. PICK's positioning in the field is as a system that brings graph learning together with graph convolution under a shared supervised objective, reducing reliance on manually defined edge rules by learning continuous edge weights from labelled data.

SPADE [12] formulates the same overall problem differently. It takes OCR tokens together with their 2D coordinates as input and treats document IE as *spatial dependency parsing* over the token stream rather than as sequence tagging. The model predicts head–tail dependency edges between tokens together with relation labels — including representation, grouping, and sub-grouping — that name the kind of structural relationship each predicted edge stands for. The predicted edges are then decoded into a structured document graph through a procedure that resolves competing edge predictions, including the tail-collision-avoidance step already referenced in §1.5; the structured target output is recovered from this parsed dependency graph rather than from token-level tag predictions. SPADE is supervised end-to-end on labelled data in the target schema. It contrasts with PICK in two ways: the graph is produced by an edge-prediction module rather than as an adjacency learned to feed graph convolution, and it is consumed by a parsing-and-decoding procedure rather than by graph-aware feature propagation.

FormNet [39] responds to a different motivating problem within the same research line: the observation that OCR tokens cannot always be reliably serialised into the linear reading order that sequence-tagging models assume, particularly on dense forms where rows, columns, and side panels disrupt the dominant reading direction. FormNet's architectural response is structural augmentation of an otherwise transformer-based document model. *Rich Attention* introduces an attention mechanism that conditions on the spatial relationships between tokens in addition to their content features, so that attention is computed in a layout-aware rather than purely content-aware manner. *Super-Tokens* are graph-convolution-derived contextual tokens constructed from each token's neighbours before the transformer's self-attention layers operate, providing each position with a structurally-informed local context that compensates for the unreliable serial order. FormNet is trained and evaluated on form-understanding benchmarks such as FUNSD and CORD. Together, these systems show three ways graph structure enters supervised KIE: as a learned weighted graph consumed by graph convolution, as a predicted dependency graph decoded into structured output, and as graph-aware structural encoding inside a transformer-based document model.

2.4.3 Where this thesis sits: deterministic graph association without supervised graph learning

The thesis's pipeline can be compared with the three §2.4.2 systems along the same axes §2.4.1 introduced. On edge production, PICK, SPADE, and FormNet all produce edges through learned modules trained end-to-end on labelled documents — PICK as an adjacency representation whose pairwise weights are fitted from data, SPADE as a head–tail dependency edge predicted token-by-token, FormNet as part of the structural augmentation that feeds its transformer en-

coder. The thesis instead constructs typed edges deterministically from geometric primitives and semantic role assignments — SAME_ROW, SAME_COL, and CONTEXT_SCOPE, with continuous weights derived from spatial distance in the local coordinate frame of each structural unit, as §1.5 describes. On downstream graph use, the three systems consume their respective graphs through learned operators: PICK through graph convolution that propagates information across edges, SPADE through a parsing-and-decoding procedure that recovers the structured target from predicted dependencies, FormNet through Rich-Attention layers that condition transformer attention on graph-derived spatial encoding. The thesis consumes its graph through a hybrid matcher built from operators that have no learned parameters: greedy bipartite matching for typical cases, structural propagation for the one-to-many associations bipartite matching cannot cover, and a rank-under-context fallback for cases where geometric matching has failed. On supervision, all three systems are trained end-to-end on labelled examples in their respective target schemas, while the thesis uses no labelled target-schema data — the entire graph construction, edge weighting, and matching machinery is determined by domain rules and geometric primitives.

What the thesis takes from this research line is its representational substrate: the typed-edge document graph, the principle that document IE benefits from graph reasoning rather than from sequence tagging alone, and specific architectural ideas including the soft-adjacency framing of PICK and the tail-collision avoidance of SPADE that §1.5.5 named at the primitive level. What the thesis leaves behind is the supervised learning machinery — both the labelled-data dependency that learning continuous edge weights or dependency predictions requires, and the graph-to-output prediction heads that translate graph structure into structured output through trained parameters. In the terms of §2.1.3, graph-based KIE is the research line whose representation this thesis *retains and re-implements without supervised training*: it keeps the graph-based view of document IE, but replaces supervised graph learning with deterministic construction and matching.

2.5 Zero-shot information extraction and semantic matching

2.5.1 Zero-shot IE as a research line

Zero-shot information extraction, in the sense relevant to this chapter, is the family of methods that operate without labelled target-schema training data. The supervised regime of §2.1.2 presupposes labelled examples in the target schema and adapts a pretrained model to them; the supervised graph-KIE systems of §2.4 depend on labelled examples for edge production and downstream graph use; the alternative paradigms of §2.2 and §2.3 either change the extraction target or remove the staging that makes modular extraction possible, but in their dominant form also depend on supervised training in some way. The zero-shot family is defined in opposition to that dependence. Its methods produce structured output without fitting parameters to labelled examples in the target schema, and their generalisation comes from prior knowledge brought into the inference setting from elsewhere. §1.4.5 already named three of these prior-knowledge sources; this section restates the framing and extends it with a fourth.

The first face is *rule-based zero-shot extraction*: no parameters are fitted from labelled data, and the prior knowledge is what the developer encodes in lexicons, regular expressions, and

decision rules — the route the thesis's default classifier takes. The second is *schema-flexible learned NER*: a model trained on cross-domain NER data is applied to a new schema by supplying the entity types as natural-language prompts at inference time, with prior knowledge transferred from the training distribution to the new schema rather than fitted to it. The third is *embedding-prototype matching*: a pre-trained multilingual sentence encoder produces vector representations of candidate strings and developer-written category prototypes, and classification is performed by cosine similarity, with prior knowledge drawn from the encoder's pretraining geometry. The fourth, which extends §1.4.5's framing, is *LLM-prompted text-to-structure extraction*: a generative language model trained on broad text-and-instruction data is prompted with a schema specification and asked to emit structured records directly from input text, with prior knowledge drawn from generative pretraining and instruction-tuning combined. The four faces correspond to four different sources of prior knowledge — encoded rules, cross-task transfer, pretrained embedding geometry, and generative-LM instruction-following — and the rest of this section uses that distinction to organise the comparison. The important point for this chapter is that all four operate without labelled examples in the target schema, but they make different bets about what kind of prior knowledge is most useful, and the choice between them is therefore a methodological commitment rather than a uniform technique.

2.5.2 Representative zero-shot IE families

Because rule-based zero-shot extraction is the thesis's default mechanism and was described in §1.4.2 and §1.4.5, this subsection focuses on the three non-default zero-shot families against which that default is positioned.

The first non-default family is *schema-flexible learned NER*, whose canonical representative is **GLiNER** [10]. The model is a compact bidirectional transformer encoder trained on a broad corpus of supervised NER data, with the distinguishing architectural commitment that the target entity types are supplied at inference time as natural-language strings rather than as a fixed set of output classes baked into a classifier head. The encoder consumes the input text together with the entity-type prompts, and a span-classification head scores candidate spans against each prompted type; a span is committed to a type when its score exceeds a threshold and the type is the argmax over the candidates the model has been asked to consider. Because the entity types are passed in at inference, the schema can be changed by changing the prompts: a new entity type — *supplement nutrient name*, *unit of measurement*, *reference-base context* — can be supplied without retraining. GLiNER and later schema-flexible NER variants share the same basic design commitment: the entity schema is supplied at inference time rather than fixed by a task-specific classifier head. The family's prior-knowledge source is *cross-task transfer*: the entity-recognition task has been learned across many schemas during training, and the family's generalisation to a new schema rests on the assumption that the task transfers without requiring the new schema to have been seen.

The second non-default family is *embedding-prototype matching*. Its representative system in this thesis is **BGE-M3** [17], a multilingual sentence encoder that produces dense vector representations across more than a hundred languages and combines dense, sparse, and multi-vector retrieval functionality at the encoder level. Within the family, classification is performed by encoding both the candidate input and a developer-written prototype pool through the en-

coder, computing cosine similarity between the candidate and each prototype, and assigning the candidate to the category whose closest prototype clears a similarity threshold and a margin against competing categories. The same matching pattern is older than BGE-M3 itself: **Sentence-BERT** [16] established the use of Siamese sentence encoders for efficient semantic similarity, and the prototype-matching mechanism builds directly on that recipe. What distinguishes BGE-M3 within the family is its scale and multi-granularity rather than the matching mechanism itself. The family's prior-knowledge source is the *pretrained embedding geometry* of the encoder: the assumption is that the encoder's pretraining distribution has produced a vector space in which semantically similar strings sit close together regardless of language, surface form, or schema, against which developer-written prototype pools serve as anchors for application-level categories.

The third non-default family is *LLM-prompted text-to-structure extraction.* The family takes an instruction-tuned generative language model and prompts it with a schema specification together with the input text, asking the model to emit structured output — typically JSON-formatted records conforming to the prompted schema — directly. The model is not fine-tuned on the target schema; the schema is communicated at inference time through the prompt, and the model's behaviour relies on its instruction-following capability and on its general capacity to produce structured output in response to a structured request. Representative systems include open-weight families such as the **Qwen-2.5** family [40] and the **Llama** family [41], together with proprietary systems such as the GPT-4 family and the Claude family, all of which can be prompted to extract structured records from OCR text given a sufficiently specified schema in the prompt. The family's prior-knowledge source is the combination of generative-language-model pretraining on broad text corpora and instruction-following capability acquired through supervised fine-tuning and reinforcement learning from human feedback in the general-LLM training pipeline. This discussion concerns LLMs used as extractors, in which the model itself produces the target structured records from input text. The use of LLMs as evaluators of records produced by another system — the role assigned to the LLM judge in §1.7.4 — belongs to evaluation methodology rather than to the related extraction paradigms surveyed here.

2.5.3 Where this thesis sits

The thesis's engagement with the three non-default zero-shot families surveyed in §2.5.2 is differential: two were evaluated as candidate replacements for the rule-based default, the third was not adopted as an extraction mechanism. For schema-flexible learned NER, the thesis evaluated GLiNER as a candidate replacement for the rule-based classifier of §1.4.2, treating it as a plausible alternative whose cross-task transfer might recover nutrient names, units, or context labels that lexicon-based rules miss; the rule-based default was retained. For embedding-prototype matching, the thesis evaluated BGE-M3-based prototype matching with the threshold and margin parameters introduced in §1.6 as an alternative classifier path, treating the encoder's pretrained embedding geometry as a candidate substitute for the rule-based decision rule; the rule-based default was retained. For LLM-prompted text-to-structure extraction, the thesis did not adopt the family as its extraction mechanism. Three reasons are sufficient at the related-work level: it weakens *determinism*, because the output varies across runs, model

versions, and prompt formulations; it weakens *stage-level control*, because a single generation collapses OCR, classification, and association into one opaque step; and it weakens *auditability*, because the four diagnostic CSVs of §1.7.5 have no analogue in a generative-LLM extraction pass. These concerns mirror the modularity and reproducibility concerns raised for frontier-VLM extractors in §2.3.3, but here they apply to text-only LLM extraction after OCR rather than to image-based VLM extraction. LLMs are used in the evaluation pipeline as judges per §1.7.4; they are not used as extractors. The empirical comparison among the rule-based default, GLiNER, and the embedding-prototype alternative is treated in the experimental chapter.

In the terms of §2.1.3, zero-shot information extraction is the research line that *operates without labelled target-schema data* — the third of the three structural responses §2.1.3 enumerated. The thesis sits squarely within this research line: its rule-based default and its evaluated alternatives all satisfy the no-labelled-target-schema-data condition, with the variation across them being the source of prior knowledge each method draws on. What this subsection establishes about the thesis's choice is not that the rule-based default is intrinsically superior to the alternatives — it is that, given the constraint set §1.1 and §2.1.3 named (zero-shot operation, layout-flexibility within domain, modular and interpretable design), the rule-based default and the evaluated alternatives all fit the constraint set, while the LLM-prompted family does not. The thesis selects within the zero-shot research line the mechanism that best preserves modularity and deterministic control.

2.6 Nutrition-label and food-data extraction

2.6.1 Prior work specifically on nutrition-label extraction

Compared with the benchmark-driven literature on forms, receipts, and tables, the literature on nutrition-label extraction is comparatively sparse and more application-oriented than benchmark-oriented. As §2.1.1 noted, nutrition and supplement labels are not part of the canonical benchmark set around which document KIE has largely organised itself, and the literature on this domain is therefore smaller and less leaderboard-driven than the work surveyed in §2.1.2 and §2.4. The work that does exist tends to combine OCR with downstream rules or, more recently, with LLM-based post-processing, in pipelines targeted at specific extraction goals — recovering nutrient names and values, normalising units, populating downstream databases — rather than at general benchmark performance. A recent example is the work of Assiri et al. on extracting nutritional elements and values from bilingual food labels using large language models [42], which reflects the broader trajectory in document IE toward OCR-and-LLM hybrids and applies that trajectory specifically to nutrition labels in a multilingual setting.

What existing nutrition-label extraction work typically targets is the recovery of visible nutrient names together with their associated quantities and, in many cases, unit normalisation; the extraction goal is the data displayed on the label, often with limited handling of multilingual surface forms or of the reference-base context under which a value is reported. The thesis's target schema is narrower in some respects and wider in others: narrower because it is restricted to supplement labels rather than general food labels, wider because it requires full ⟨nutrient, quantity, unit, context⟩ tuple association in which each numerical value must be

paired with the correct unit *and* the correct reference-base context — per_100g, per_serving, per_daily_dose — across the multilingual layouts that EU supplement labelling routinely produces. The association problem, in this stronger sense — pairing each nutrient with the correct quantity, unit, and context across slash-rows, multi-context columns, and paragraph-style declarations — appears not to have been a central evaluation target in the published nutrition-label work to date. Adjacent OCR-on-packaging work and food-product databases provide infrastructure that the supplement-label setting can draw on, but they are not direct precedent for the specific tuple-association task that defines this thesis; §2.6.2 takes them up next.

2.6.2 Adjacent food-data and retail-packaging extraction

The most direct neighbour of nutrition-label extraction is OCR work on food packaging more generally. **HalalBench** [43] is a recent open multilingual OCR benchmark for food-packaging ingredient lists, motivated explicitly by the lack of standard OCR benchmarks for food packaging and by the practical challenges the domain presents — curved surfaces, dense multilingual text, and sub-8pt fonts of the kind supplement labels routinely use. Comparative OCR studies have evaluated production engines including Tesseract, EasyOCR, PaddleOCR, and TrOCR on real-world food-packaging images that include both ingredient lists and nutrition-facts panels [44], providing applied OCR-engine comparisons on this kind of imagery. The role of this work for the thesis is at the perceptual layer rather than at the extraction-target layer: these studies address the perceptual layer of food-data extraction, but not the downstream nutrient–quantity–unit–context association problem targeted here.

A second strand of adjacent work consists of food-product databases that organise nutrition data at scale without claiming to be complete extraction architectures. **Open Food Facts** [45] is a major crowd-contributed food-product and nutrition-data resource: a multilingual database of food products including ingredients, allergens, and nutrition facts, populated through a combination of crowd contribution, automatic extraction, and curation. The Open Food Facts project also maintains an active interest in automatic nutrition-facts extraction from photographs, which indicates that automatic extraction from nutrition-facts images is an active applied problem outside the academic literature. **FLIP** [46] is an analogous database focused on Canadian packaged foods, providing label-data resources for downstream use. These databases support the domain motivation for nutrition-data extraction at a scale academic benchmarks do not yet cover, but they are data resources rather than extraction systems, and they do not constitute precedent for the specific zero-shot tuple-association task this thesis targets.

2.6.3 The gap this thesis addresses

The first version of the gap is at the benchmark level. No widely adopted canonical benchmark comparable to FUNSD, SROIE, CORD, or PubTabNet exists for the specific supplement-label tuple-association task that this thesis targets. The consequence is that published nutrition-label work, including the recent OCR-and-LLM hybrids §2.6.1 surveyed and the food-packaging OCR studies §2.6.2 cited, evaluates against bespoke targets defined within each study rather than against a shared benchmark, which makes cross-system comparison difficult and the leaderboard-driven progress that §2.1.1 described unavailable in the way it has been for forms,

receipts, and tables.

The second and more substantive version of the gap is at the level of combined constraints. The gap is not that nutrition labels have been ignored, but that existing work rarely combines the four constraints this thesis places at the centre: *zero-shot operation*, *multilingual supplement-label vocabulary*, *explicit nutrient–quantity–unit–context association under multiple reference bases*, and an *interpretable modular pipeline whose intermediate stages can be inspected.* Each of the four constraints is addressed individually somewhere in the surveyed literature — zero-shot operation in the research line surveyed in §2.5, multilingual OCR on packaging in the comparative work cited in §2.6.2, the relational-association problem implicitly addressed by the graph-KIE family surveyed in §2.4, and modular cascaded pipelines in the OCR-dependent paradigm surveyed in §2.3.1 — but their combined application to the supplement-label setting under EU labelling conventions is not directly supported by any prior system or benchmark the chapter has identified. The gap that this thesis addresses is therefore the combined application of these constraints to the supplement-label setting.

2.7 Positioning of this thesis

The thesis's location across the three structural-response categories of §2.1.3 can be stated as a configuration of choices. The thesis does not *change the extraction target*: it does not adopt the table-structure-recognition path of §2.2, which would reconstruct a grid before tuple extraction. Nor does it remove the staged OCR-to-reasoning architecture: it does not adopt the OCR-free or vision-language document-understanding path of §2.3. The thesis *retains the graph representation but re-implements it without supervised training*: it borrows architectural ideas from the graph-based KIE family of §2.4 — typed-edge document graphs, soft adjacency, tail-collision avoidance — and constructs them deterministically from geometric primitives and semantic role assignments rather than learning them from labelled data. The thesis *operates without labelled target-schema data*: it sits within the zero-shot information-extraction research line of §2.5, with its rule-based default selected against GLiNER-style schema-flexible NER and BGE-M3-based embedding-prototype matching as evaluated alternatives, and with LLM-prompted text-to-structure extraction excluded as the extraction mechanism. This three-part restatement is the core summary of where the thesis sits in the surveyed landscape.

The same positioning can be stated as a response to the constraint set §1.1 introduced and §2.1.3 restated: zero-shot operation, layout-flexibility within the supplement-label domain, and modular and interpretable design, with §2.6.3 adding the domain-specific commitment to nutrient–quantity–unit–context tuple association under multiple reference bases. Each of the surveyed alternatives, considered against this constraint set, leaves at least one of its constraints unresolved. TSR leaves the target-schema constraint unresolved, because it reconstructs a grid rather than the required tuples and would still require a downstream typing stage. OCR-free document models leave the zero-shot constraint unresolved, because their reliable schema-conformant operation depends on supervised adaptation to the target schema. Frontier-VLM extraction leaves the modularity and auditability constraints unresolved, because end-to-end generation hides the intermediate decisions a modular pipeline exposes. Supervised graph-KIE leaves the labelled-data constraint unresolved, because its edge production

and graph-to-output prediction are fitted to labelled examples in the target schema. LLM-prompted text-to-structure extraction leaves determinism, stage-level control, and auditability unresolved for the same architectural reasons that frontier-VLM extraction does, applied at the text-only layer after OCR. The thesis's deterministic OCR-then-rules-then-graph configuration is not presented as universally strongest on every individual criterion; it is the configuration the chapter has identified as satisfying the constraint set as a whole.

This is the chapter's closing claim. The surveyed research lines have each contributed to one or more of the constraints in isolation, and the thesis borrows from several of them — soft adjacency from PICK, tail-collision avoidance from SPADE, the broader graph-as-document-representation principle, multilingual sentence-encoder geometry for the embedding-classifier ablation, and the post-OCR repair tradition of cascaded document-IE pipelines — but no single prior system surveyed in this chapter combines the four constraints in the configuration the supplement-label setting requires. The thesis's configuration is justified not by claiming universal superiority over the alternatives, each of which is stronger on its own optimisation axis, but by being the configuration this chapter has identified as fitting the four constraints in combination.

Bibliography

[1] S. Long, X. He, and C. Yao, "Scene text detection and recognition: The deep learning era," *International Journal of Computer Vision (IJCV)*, vol. 129, pp. 161–184, 2021.

[2] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in *Proceedings of CVPR*, 2019.

[3] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarisation," in *Proceedings of AAAI*, 2020.

[4] X. Zhou, C. Yao, H. Wen, Y. Wang, S. Zhou, W. He, and J. Liang, "EAST: An efficient and accurate scene text detector," in *Proceedings of CVPR*, 2017.

[5] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, vol. 39, no. 11, pp. 2298–2304, 2017.

[6] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proceedings of ICML*, 2006.

[7] F. Sheng, Z. Chen, and B. Xu, "NRTR: A no-recurrence sequence-to-sequence model for scene text recognition," in *Proceedings of ICDAR*, 2019.

[8] Y. Du, Z. Chen, C. Jia, X. Yin, T. Zheng, C. Li, Y. Du, and Y.-G. Jiang, "SVTR: Scene text recognition with a single visual model," in *Proceedings of IJCAI*, 2022.

[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

[10] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proceedings of NAACL*, 2024.

[11] W. Yu, N. Lu, X. Qi, P. Gong, and R. Xiao, "PICK: Processing key information extraction from documents using improved graph learning-convolutional networks," in *Proceedings of ICPR*, 2020.

[12] W. Hwang, J. Yim, S. Park, S. Yang, and M. Seo, "SPADE: Spatial dependency parsing for semi-structured document information extraction," *Proceedings of ACL Findings*, 2021.

[13] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.

[14] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," *arXiv preprint arXiv:1301.3781*, 2013.

[15] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proceedings of EMNLP*, 2014, pp. 1532–1543.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of EMNLP-IJCNLP*, 2019.

[17] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, 2024.

[18] G. Jaume, H. K. Ekenel, and J.-P. Thiran, "FUNSD: A dataset for form understanding in noisy scanned documents," in *Proceedings of ICDAR Workshops*, 2019.

[19] Z. Huang, K. Chen, J. He, X. Bai, D. Karatzas, S. Lu, and C. V. Jawahar, "ICDAR2019 competition on scanned receipt OCR and information extraction," in *Proceedings of ICDAR*, 2019.

[20] S. Park, S. Shin, B. Lee, J. Lee, J. Surh, M. Seo, and H. Lee, "CORD: A consolidated receipt dataset for post-OCR parsing," in *Workshop on Document Intelligence at NeurIPS*, 2019.

[21] M. Li, Y. Xu, L. Cui, S. Huang, F. Wei, Z. Li, and M. Zhou, "DocBank: A benchmark dataset for document layout analysis," in *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, 2020.

[22] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in *Proceedings of KDD*, 2020.

[23] Y. Xu, Y. Xu, T. Lv, L. Cui, F. Wei, G. Wang, Y. Lu, D. Florencio, C. Zhang, W. Che *et al.*, "LayoutLMv2: Multi-modal pre-training for visually-rich document understanding," in *Proceedings of ACL-IJCNLP*, 2021.

[24] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proceedings of ACM MM*, 2022.

[25] S. Appalaraju, B. Jasani, B. U. Kota, Y. Xie, and R. Manmatha, "DocFormer: End-to-end transformer for document understanding," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[26] R. Powalski, Ł. Borchmann, D. Jurkiewicz, T. Dwojak, M. Pietruszka, and G. Pałka, "Going full-TILT boogie on document understanding with text-image-layout transformer," in *International Conference on Document Analysis and Recognition (ICDAR)*, 2021.

[27] T. Hong, D. Kim, M. Ji, W. Hwang, D. Nam, and S. Park, "BROS: A pre-trained language model focusing on text and layout for better key information extraction from documents," in *Proceedings of AAAI*, 2022.

[28] X. Zhong, E. ShafieiBavani, and A. Jimeno-Yepes, "Image-based table recognition: Data, model, and evaluation," in *European Conference on Computer Vision (ECCV)*, 2020, arXiv preprint 2019; published at ECCV 2020. Introduces PubTabNet and the TEDS metric.

[29] B. Smock, R. Pesala, and R. Abraham, "GriTS: Grid table similarity metric for table structure recognition," in *International Conference on Document Analysis and Recognition (ICDAR)*, 2023.

[30] A. Nassar, N. Livathinos, M. Lysak, and P. Staar, "TableFormer: Table structure understanding with transformers," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[31] C. Cui, T. Lyu, W. Zhang, X. Yan *et al.*, "PaddleOCR 3.0 technical report," *arXiv preprint arXiv:2507.05595*, 2025.

[32] PaddleOCR Team, "PP-StructureV3 documentation," https://github.com/PaddlePaddle/PaddleOCR, 2025, documentation for PP-StructureV3 hierarchical document parsing pipeline, including SLANet/SLANeXt table-recognition modules. Verify exact URL/path against the version of the docs you consulted.

[33] X. Zheng, D. Burdick, L. Popa, X. Zhong, and N. X. R. Wang, "Global table extractor (GTE): A framework for joint table identification and cell structure recognition using visual context," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2021, introduces FinTabNet.

[34] M. Li, L. Cui, S. Huang, F. Wei, M. Zhou, and Z. Li, "TableBank: Table benchmark for image-based table detection and recognition," in *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC)*, 2020.

[35] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, "OCR-free document understanding transformer," in *Proceedings of ECCV*, 2022.

[36] K. Lee, M. Joshi, I. Turc, H. Hu, F. Liu, J. M. Eisenschlos, U. Khandelwal, P. Shaw, M.-W. Chang, and K. Toutanova, "Pix2Struct: Screenshot parsing as pretraining for visual language understanding," in *Proceedings of ICML*, 2023.

[37] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, "Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond," *arXiv preprint arXiv:2308.12966*, 2023.

[38] Z. Chen, J. Wu, W. Wang, W. Su, G. Chen, S. Xing, M. Zhong, Q. Zhang, X. Zhu, L. Lu *et al.*, "InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks," *Proceedings of CVPR*, 2024.

[39] C.-Y. Lee, C.-L. Li, T. Dozat, V. Perot, G. Su, N. Hua, J. Ainslie, R. Wang, Y. Fujii, and T. Pfister, "FormNet: Structural encoding beyond sequential modeling in form document information extraction," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[40] A. Yang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024, verify full author list against the published technical report.

[41] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023, verify full author list against the published paper.

[42] F. Y. Assiri, M. D. Alahmadi, M. A. Almuashi, and A. M. Almansour, "Extract nutritional information from bilingual food labels using large language models," *Journal of Imaging*, vol. 11, no. 8, p. 271, 2025.

[43] HalalBench Authors, "HalalBench: An open multilingual OCR benchmark for food-packaging ingredient lists," Verify exact author list, venue, and arXiv/DOI identifier from the source you consulted., 2026, cited in 3.6.2 as a recent multilingual OCR benchmark for food-packaging ingredient lists.

[44] Nagayi *et al.*, "Comparative evaluation of OCR engines on food-packaging images," Verify full author list, exact title, venue, and identifier from the source you consulted., 2025, cited in 3.6.2 as evaluating Tesseract, EasyOCR, PaddleOCR, and TrOCR on real-world food-packaging images including ingredient lists and nutrition-facts panels.

[45] Open Food Facts, "Open food facts: A free, open, collaborative database of food products from around the world," https://world.openfoodfacts.org/, 2024.

[46] M. Ahmed *et al.*, "The food label information program (FLIP): A database of canadian packaged foods," *Frontiers in Nutrition*, 2022, verify full author list, volume, page numbers, and DOI from the Frontiers paper.